

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_5de56b64-647\agent-ae999402d9859281d.jsonl`  
- SHA-256: `0b4f4601d805d762a5b6b0865d32b111d3ab4a1d97e96300c205b5c1e4cb3f4`  
- Source modified: `2026-06-21T14:39:19+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_5de56b64-647`  
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`
```

Transcript

1. user (2026-06-21T14:36:44.487Z)

Repo: C:\Users\avidu\Projects\badminton-highlight-indexer (the engine). Map the Cloud Run deploy story precisely. The goal: a PERSISTENT, internet-reachable FastAPI engine ENDPOINT that a hosted SPA can call (POST /api/process, GET /api/jobs, /api/compile, /api/highlights). Read deploy/cloudrun/** (Dockerfile, README, any deploy scripts), deploy/gcp/**, backend/main.py (the --server entrypoint / FastAPI app), backend/compute/** (the M6 ComputeBackend / CloudRunCompute), and backend/worker/**.

Determine:

- Does deploy/cloudrun/ build/deploy the FastAPI SERVICE (long-lived HTTP endpoint), or only the per-request Job WORKER, or both? Cite file:line.
- What exactly is missing to stand up the FastAPI app as a Cloud Run *service* reachable over HTTPS (a service Dockerfile, a 'gcloud run deploy --port' invocation, PORT env handling in main.py, healthcheck)? Cite evidence.
- Does main.py honor a PORT env var and 0.0.0.0 bind for Cloud Run? Cite file:line.
- Is async job state in-process (in-memory dict) or persistent (SQLite)? A scale-to-zero / multi-instance Cloud Run service needs durable job state ? flag if jobs would be lost. Cite the job store in backend/main.py.

Classify each gap as claude-doable-no-spend (code/Dockerfile/script) vs owner-gated (actual gcloud deploy + GCP spend).

2. assistant (2026-06-21T14:36:46.667Z)

I'll systematically map the Cloud Run deploy story for the badminton-highlight-indexer. Let me start by exploring the directory structure and key files.

3. user (2026-06-21T14:36:47.987Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/.github/workflows/ci.yml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/reporting/spec.yaml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/.github/workflows/ci.yml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/backend/reporting/spec.yaml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/.github/workflows/ci.yml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/backend/reporting/spec.yaml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/.github/workflows/ci.yml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/backend/reporting/spec.yaml  
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/.github/workflows/ci.yml
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/backend/reporting/spec.yaml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-stitch/.github/workflows/ci.yml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-stitch/backend/reporting/spec.yaml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/recdoc/.github/workflows/ci.yml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/recdoc/backend/reporting/spec.yaml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/resweep/.github/workflows/ci.yml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/resweep/backend/reporting/spec.yaml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier1-windowing/.github/workflows/ci.yml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/tier1-windowing/backend/reporting/spec.yaml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/w2-safeguard/.github/workflows/ci.yml
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/w2-safeguard/backend/reporting/spec.yaml
```

4. assistant (2026-06-21T14:36:49.710Z)

Let me search for deploy directories more specifically:

5. user (2026-06-21T14:36:50.696Z)

```
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:59 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:59 ..
drwxr-xr-x 1 avidu 197609    0 Jun 20 21:27 .claude
drwxr-xr-x 1 avidu 197609    0 Jun 21 20:04 .git
drwxr-xr-x 1 avidu 197609    0 Jun 10 11:31 .github
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun  8 07:11 .sixth
drwxr-xr-x 1 avidu 197609    0 Jun 14 01:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jun 20 23:51 artifacts
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:31 backend
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:11 deploy
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:54 docs
drwxr-xr-x 1 avidu 197609    0 Jun 19 21:21 eval_baselines
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:20 output
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:14 scratch
drwxr-xr-x 1 avidu 197609    0 Jun 20 13:48 scripts
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:31 tests
drwxr-xr-x 1 avidu 197609    0 Jun 19 14:26 tools
drwxr-xr-x 1 avidu 197609    0 Jun 10 15:57 training
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:30 videos
```

6. assistant (2026-06-21T14:36:52.041Z)

Now let me explore the deploy directory:

7. user (2026-06-21T14:36:53.217Z)

```
total 20
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:11 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:59 ..
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:20 cloudrun
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:53 digitalocean
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:25 gcp
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\Dockerfile
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\cloudrun\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\digitalocean\bootstrap.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\digitalocean\gpu_setup.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\digitalocean\mount_scratch.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\digitalocean\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\create_gpu_vm.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\ops-agent-startup.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\provision.sh
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\deploy\gcp\teardown.sh
```

8. assistant (2026-06-21T14:36:54.876Z)

Now let me read the key files. I'll start with the CloudRun deployment files and main.py:

9. user (2026-06-21T14:36:55.543Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
```

```

35      && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36      && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37      && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38      && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39      && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40      torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41      && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42      && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43      hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45      # --- Main application env ---
46      WORKDIR /app
47      COPY pyproject.toml README.md ./
48      COPY backend ./backend
49      COPY training ./training
50      RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52      # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53      # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54      ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55      WASB_REPO_DIR=/opt/models/WASB-SBDT \
56      WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57      RALLY_DETECTOR_IMPL=native
58
59      # The dispatcher (CloudRunCompute) supplies the per-execution args:
60      #   infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61      ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

10. user (2026-06-21T14:36:56.166Z)

```

1      # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3      The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4      dispatches a Cloud Run Job execution running this image's worker on an L4, then
bucket-polls
5      a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7      > Status: the dispatch shim + image are landed + mock-tested (M6). The real
build + push +
8      > run on an L4 is the M7 owner step, within the $100/?10k budget (D17) ?
confirm cmd + hourly
9      > cost first, delete every resource after ([gcp-vm-always-delete]). CI does
not build this image.
10
11     ## 0. Prerequisites (owner)
12     - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13     - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14     `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15     - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17     ## 1. Build + push the image (Artifact Registry)
18     ``bash

```

```

19     PROJECT=<gcp-project>; REGION=asia-south1; REPO=rally; IMG=rally-worker
20     gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
21     gcloud builds submit --tag $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest -f
deploy/cloudrun/Dockerfile .
22     ```
23     *(A CUDA + conda image is large; the first build is slow. Budget-watch per D17.)*
24
25     ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
26     ```bash
27     gcloud run jobs create rally-worker \
28         --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
29         --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
30         --max-retries 0 --task-timeout 3600 \
31         --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
32     # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
33     ```
34
35     ## 3. Wire the dispatcher (control plane)
36     The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
37     `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
38     ```bash
39     export DEPLOYMENT_PROFILE=cloud-serving
40     export CLOUD_RUN_JOB=rally-worker CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
41     # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
42     # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
43     python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
44     ```
45
46     ## 4. M7 acceptance (the owner runlog)
47     Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
48     `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
49     rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
50     exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
51
52     ## Gotchas
53     - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
54     + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
55     - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
56     ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
57     - A **failed** Job currently writes **no** marker ? the dispatcher's bounded poll times
out
58     (`ExecutionPolicy.max_wait_sec`) and `_dispatch_remote` returns a clean **rc 4** (not
a traceback).
59     A failure-marker (carrying the real non-zero rc, so the dispatcher distinguishes
**failed** from
60     *hung/cold-start-slow*) is a follow-up.
61     - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
62

```

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException, Request
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )
39    from backend.results import Failure # standardized error/result contract
40    from backend.reporting import emit_indexer_report
41    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42    from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43    from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44
45    app = FastAPI(title="Sports Highlight Indexer API")
46
47
48    def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
49        """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
50        default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
51        does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
52        src = os.environ if env is None else env
53        raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
54        origins = [o.strip() for o in raw.split(",") if o.strip()]
55        return origins or ["*"]
56
57
58    # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by

```

```

browsers per the
59     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
60     # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can
61     # lock it to its origin; the middleware is fixed at startup.
62     app.add_middleware(
63         CORSMiddleware,
64         allow_origins=_cors_origins_from_env(),
65         allow_credentials=False,
66         allow_methods=["*"],
67         allow_headers=["*"],
68     )
69
70     # Serves static frontend files directly (now under api/ per approved structure)
71     os.makedirs("backend/api/static", exist_ok=True)
72     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
73
74     @app.get("/", response_class=HTMLResponse)
75     def serve_index():
76         index_path = "backend/api/static/index.html"
77         if os.path.exists(index_path):
78             with open(index_path, "r", encoding="utf-8") as f:
79                 return f.read()
80         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
81
82     # Global variables initialized at startup
83     db_instance: Optional[Database] = None
84     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
85
86     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
87     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
88     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
89     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
90     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
91     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
92     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
93     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
94         JobWaiting,
95         _arm_wake_timer,
96         _drain_due_jobs,
97         _get_executor,
98         _job_to_request,
99         _MAX_DRAIN_WAKE_SEC,
100         _maybe_record_job_cost,
101         _run_claimed_job,
102         _schedule_next_drain_if_waiting,
103     )
104
105     # Legacy thin wrapper for any remaining direct calls during transition.
106     # New code should import load_app_config / AppConfig from backend.config directly.
107     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
108         cfg = load_app_config(config_path)
109         return cfg.model_dump(mode="json")
110
111     # --- API Models ---
112     class ProcessRequest(BaseModel):
113         video_path: str
114         player_pool: Optional[List[str]] = None
115         max_frames: Optional[int] = None
116         segmenter: Optional[str] = None

```

```

117
118 class QueryRequest(BaseModel):
119     video_id: str
120     server: Optional[str] = None
121     receiver: Optional[str] = None
122     duration_min: Optional[float] = None
123     event_type: Optional[str] = None
124
125 class CompileRequest(BaseModel):
126     video_id: str
127     segment_ids: List[int]
128     output_filename: str
129
130 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
131     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
132
133     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
134     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
135     and
136     keep the prior good results intact ? a failed/empty re-run never destroys prior
137     data.
138     """
139     if segments:
140         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
141     else:
142         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
143
144 # --- API Endpoints ---
145 @app.get("/api/config")
146 def get_config():
147     return global_config
148
149 @app.get("/api/health")
150 def health():
151     """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
152     (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
153     confirm the engine is reachable and which detector is wired. Never raises."""
154     sport = getattr(global_config, "sport", None)
155     indexing = getattr(global_config, "indexing", None)
156     return {
157         "status": "ok",
158         "sport": sport,
159         "default_segmenter": getattr(indexing, "default_segmenter", None),
160     }
161
162 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
163     """The full hash ? validate ? segment ? report pipeline for one video.
164
165     This is the former synchronous /api/process body, unchanged in behavior, now run on
166     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
167     the sync endpoint used to return (UI compatibility); the per-video observability
168     report is still emitted in `finally:` and grafted as response["reports"].
169
170     `progress` is an optional callable(stage: str) used to surface coarse job progress.
171     Raises on unexpected internal errors ? the caller records those as a job failure
172     (after this function has already restored the pre-run segments, see below).
173     """
174     notify = progress or (lambda stage: None)
175
176     notify("hashing")
177     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
178     (identity ?
179     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
180     original

```

```

178         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
179         # consumers (hash / validators / segmenter) use the resolved local path.
180         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
181         video_id = compute_video_id(local_video)
182         was_reingest = db_instance.get_video(video_id) is not None
183
184         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
185         notify("validating")
186         logger.info(f"Running validations for {req.video_path}...")
187         val_results, val_context = registry.run_all_with_context(local_video, global_config)
188         failures = [r for r in val_results if not r.passed]
189
190         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
191         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
192         seg_name = req.segmenter or default_seg
193         segmenter_actual = None
194         segmentation_ran = False
195         response: Optional[Dict[str, Any]] = None
196         try:
197             if failures:
198                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
199                 # status; it no longer destroys the video's prior good segments.
200                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
201                 response = {
202                     "video_id": video_id,
203                     "status": "failed",
204                     "message": "Video failed validation checks.",
205                     "results": [r.model_dump() for r in val_results],
206                 }
207                 return response
208
209             # 2. Run segmenter & indexer
210             logger.info("[API] Video passed checks. Segmenting and indexing...")
211             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
212
213             segmenter_cls = segmenter_registry.get(seg_name)
214             if not segmenter_cls:
215                 # Submit-time validation already 400s unknown names; this is the defensive
216                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
217                 available = list(segmenter_registry.list_available().keys())
218                 response = {
219                     "video_id": video_id,
220                     "status": "failed",
221                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
222                     "results": [r.model_dump() for r in val_results],
223                     "play_segments": [],
224                 }
225                 return response
226
227             logger.info(f"[API] Using segmenter: {seg_name}")
228             notify(f"segmenting ({seg_name})")
229             segmenter_actual = seg_name
230             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
231             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
232             play_wm, cand_wm = db_instance.segment_watermarks(video_id)

```

```

233         segmenter = segmenter_cls(db_instance, global_config)
234     try:
235         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
236     except Exception:
237         # A raising segmenter must not leave half-written rows: restore the pre-run
238         # state (same destroy-after-confirm contract as the Failure branch), then
239         let
240             # the job worker record the failure.
241             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
242             raise
243     segmentation_ran = True
244
245     if isinstance(result, Failure):
246         logger.error(f"[API] Segmenter failed: {result.message}")
247         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
248         response = {
249             "video_id": video_id,
250             "status": "failed",
251             "message": f"Segmenter '{seg_name}' failed: {result.message}",
252             "results": [r.model_dump() for r in val_results],
253             "play_segments": [],
254         }
255         return response
256
257     segments = result.value if hasattr(result, "value") else result
258     notify("finalizing")
259     _finalize_reingest(video_id, segments, play_wm, cand_wm)
260
261     response = {
262         "video_id": video_id,
263         "status": "success",
264         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
265         "results": [r.model_dump() for r in val_results],
266         "play_segments": segments,
267     }
268     return response
269
270 finally:
271     # Always emit the per-video observability report (next to the video, or to
272     # report.output_dir). Never raises. Surface the paths in the response.
273     paths = emit_indexer_report(
274         db=db_instance, video_id=video_id, video_path=req.video_path,
275         config=global_config,
276         val_results=val_results, val_context=val_context,
277         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
278         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
279     )
280
281     if paths and isinstance(response, dict):
282         response["reports"] = paths
283
284
285 @app.post("/api/process", status_code=202)
286 def process_video(req: ProcessRequest, request: Request):
287     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
288
289     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
290     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
291     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
292     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
293
294     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
295     single-user,
296     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
297     missing or

```

```

292         # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
293         try:
294             owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295         except edge_auth.EdgeAuthError as e:
296             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
297
298         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299         try:
300             validate_input_path(req.video_path, allowed_root=allowed_root)
301             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
304         except ValueError as e:
305             raise HTTPException(status_code=400, detail=str(e)) from e
306         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
307         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
308         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
309         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
310         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
311         # rejects a non-local URI as out-of-root.
312         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
313             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
314         if req.segmenter and not segmenter_registry.get(req.segmenter):
315             available = list(segmenter_registry.list_available().keys())
316             raise HTTPException(
317                 status_code=400,
318                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
319         )
320
321         job_id = uuid.uuid4().hex
322         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
323                                     req.player_pool, req.max_frames, owner_id=owner_id):
324             raise HTTPException(status_code=500, detail="Failed to persist job record.")
325         _get_executor().submit(_drain_due_jobs)
326         return JSONResponse(
327             status_code=202,
328             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
329         )
330
331
332         @app.get("/api/jobs/{job_id}/cost")
333         def get_job_cost(job_id: str):
334             """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
335             GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
336             null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
337             cost = db_instance.get_job_cost(job_id)
338             if cost is None:
339                 raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
340             fx = global_config.billing.fx_inr_per_usd
341             usd = cost.get("cost_usd")
342             cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
343             cost["fx_inr_per_usd"] = fx

```

```

344         return cost
345
346
347     @app.get("/api/videos/{video_id}/cost")
348     def get_video_cost(video_id: str):
349         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
350         ``total_cost_inr`` + the
351         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
352         several."""
353         agg = db_instance.get_video_cost(video_id)
354         fx = global_config.billing.fx_inr_per_usd
355         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
356         agg["fx_inr_per_usd"] = fx
357         return agg
358
359     @app.get("/api/jobs/{job_id}")
360     def get_job(job_id: str):
361         """Job state: {status, progress, result, reports}. `status` = execution state
362         (queued|running|done|failed); the pipeline verdict is result["status"]."""
363         job = db_instance.get_job(job_id)
364         if not job:
365             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
366         result = job.get("result")
367         return {
368             "job_id": job["id"],
369             "status": job["status"],
370             "progress": job.get("progress"),
371             "video_id": job.get("video_id"),
372             "error": job.get("error"),
373             "result": result,
374             "reports": (result or {}).get("reports"),
375             "created_at": job.get("created_at"),
376             "started_at": job.get("started_at"),
377             "finished_at": job.get("finished_at"),
378         }
379
380     # --- Chunked upload (B2, PR-2) -----
381     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
382     # in
383     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
384     # via
385     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
386     # sequential-part logic, and abort cleanup ? is unchanged.
387     from backend.api import uploads as uploads_api
388
389     app.include_router(uploads_api.router)
390
391     # --- Highlight download (B3, PR-2) -----
392
393     @app.get("/api/highlights/{video_id}/{filename}")
394     def download_highlight(video_id: str, filename: str):
395         """Stream a compiled highlight reel ? `filename` is the bare name given to
396         /api/compile
397         (which only writes into output_dir; the browser previously got back an unreachable
398         server-local path)."""
399         try:
400             name = safe_output_filename(filename)
401         except ValueError as e:
402             raise HTTPException(status_code=400, detail=str(e)) from e
403         if not db_instance.get_video(video_id):
404             raise HTTPException(status_code=404, detail="Indexed video record not found.")
405         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
406         or "output"

```

```

403     path = os.path.join(output_dir, name)
404     try:
405         path = resolve_under_root(path, output_dir)
406     except ValueError as e:
407         raise HTTPException(status_code=400, detail=str(e)) from e
408     if not os.path.isfile(path):
409         raise HTTPException(status_code=404,
410                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
411     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
412     return FileResponse(path, media_type=media, filename=name)
413
414
415 @app.post("/api/query")
416 def query_play_segments(req: QueryRequest):
417     filters = {
418         "server": req.server,
419         "receiver": req.receiver,
420         "duration_min": req.duration_min,
421         "event_type": req.event_type
422     }
423     segments = db_instance.query_play_segments(req.video_id, filters)
424     return {"play_segments": segments}
425
426 @app.post("/api/compile")
427 def compile_highlights(req: CompileRequest):
428     video = db_instance.get_video(req.video_id)
429     if not video:
430         raise HTTPException(status_code=404, detail="Indexed video record not found.")
431
432     # Retrieve matching play segments from db
433     all_segments = db_instance.query_play_segments(req.video_id, {})
434     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
435
436     if not matched_segments:
437         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
438
439     # Reject path traversal / absolute output names (os.path.join would otherwise let an
440     # absolute or '..'-laden output_filename write anywhere on disk).
441     try:
442         out_name = safe_output_filename(req.output_filename)
443     except ValueError as e:
444         raise HTTPException(status_code=400, detail=str(e)) from e
445
446     # The configured shot-count floor (stitching.min_shot_count) applies on the API
447     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
448     # metadata are exempt: the floor filters known counts only, so explicitly
449     # requested manual/legacy segments can't silently vanish under the default floor.
450     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
451     kept = []
452     for s in matched_segments:
453         meta = s.get("metadata") or {}
454         sc = meta.get("shot_count") if isinstance(meta, dict) else None
455         try:
456             if sc is not None and int(sc) < stitching.min_shot_count:
457                 continue
458             except (TypeError, ValueError):
459                 pass # unparseable count -> treat as unknown, keep
460             kept.append(s)
461     if len(kept) < len(matched_segments):
462         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
463                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}")

```

```

segments below the floor.")
464         if not kept:
465             raise HTTPException(status_code=400,
466                                 detail=f"All {len(matched_segments)} matching segments fall
below "
467 f"stitching.min_shot_count={stitching.min_shot_count}.")
468
469         # Extract start/end time pairs
470         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
471
472         # Run compiler with the configured stitching paddings + optional branding watermark
473         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
474         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
475                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
476         try:
477             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
478             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
479             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
480             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
481             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
482             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
483             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
484             # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
485             # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
$4.3).
486             download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
487             return {"status": "success", "filepath": out_path, "download_url": download_url}
488         except Exception as e:
489             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
490
491     # --- CLI Debug Utility & Orchestration ---
492     def run_cli_diagnose_clip(video_path: str, timestamp: float):
493         """CLI debugging utility to examine segment properties around a timestamp."""
494         print("=" * 60)
495         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
496         print("=" * 60)
497
498         cfg = load_config() # legacy wrapper still works, returns dict for now
499
500         # 1. Validation check diagnostics
501         print("[DIAGNOSTIC] Running validation framework...")
502         results = registry.run_all(video_path, cfg)
503         for r in results:
504             status_symbol = "[PASS]" if r.passed else "[FAIL]"
505             print(f" {status_symbol} {r.validator_name}: {r.message}")
506             if r.details:
507                 print(f"         Details: {r.details}")
508
509         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
510         print("-" * 60)
511         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
512         import cv2
513         cap = cv2.VideoCapture(video_path)
514         if not cap.isOpened():

```

```

515         print("[FAIL] OpenCV could not open video.")
516         return
517
518     fps = cap.get(cv2.CAP_PROP_FPS)
519     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
520
521     start_frame = max(0, int((timestamp - 3.0) * fps))
522     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
523
524     # Measure frame-by-frame changes in sample region
525     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
526     prev_gray = None
527     motions = []
528
529     for _ in range(start_frame, end_frame):
530         ret, frame = cap.read()
531         if not ret:
532             break
533         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
534         gray = cv2.GaussianBlur(gray, (21, 21), 0)
535
536         if prev_gray is not None:
537             diff = cv2.absdiff(prev_gray, gray)
538             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
539             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
540             motions.append(motion_ratio)
541             prev_gray = gray
542
543     cap.release()
544
545     if motions:
546         avg_motion = sum(motions) / len(motions)
547         # Support both legacy dict (from wrapper) and future direct model
548         if hasattr(cfg, "indexing"):
549             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
550         else:
551             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
552         print(f" Sample Frame Count: {len(motions)}")
553         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
554         if avg_motion > threshold:
555             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
556         else:
557             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
558     else:
559         print(" [ERROR] No visual frames were extracted in the sample range.")
560
561     print("=" * 60)
562
563
564     def _enforce_startup_or_exit(config) -> None:
565         """M5: run the per-profile startup checks for the server/CLI path and
`sys.exit(1)` on a
566         FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE
any work.
567         A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No
GPU probe
568         here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
569         so the fail-fast wiring + the default-OFF no-op are unit-testable."""
570         try:

```

```

571         enforce_startup_checks(config, logger=logger)
572     except StartupCheckError as e:
573         logger.error("[STARTUP] refusing to start: %s", e)
574         sys.exit(1)
575
576
577     def main():
578         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
579         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
580         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
581         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
582         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
583         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
584         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
585         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
586
587         available_segmenters = list(segmenter_registry.list_available().keys())
588         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
589         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
590         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
591         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
592
593         args = parser.parse_args()
594
595         if args.setup:
596             # Invoke the diagnostics/bootstrap script (renamed from the old root
597             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
598             # build system). Resolve its path from this file so `indexer --setup`
599             # works regardless of the caller's cwd.
600             import subprocess
601             doctor_path = os.path.join(
602                 os.path.dirname(os.path.abspath(__file__)),
603                 "scripts", "doctor.py",
604             )
605             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
606             subprocess.run([sys.executable, doctor_path])
607             return
608
609         if args.trim_start is not None and args.trim_end is not None:
610             if not args.video_path:
611                 print("[ERROR] Please provide --video-path to extract a segment.")
612                 return
613
614             # Determine output path
615             out_filename = args.trim_output or "trimmed_segment.mp4"
616             if os.path.isabs(out_filename):
617                 out_path = out_filename
618                 out_dir = os.path.dirname(out_path)
619                 out_name = os.path.basename(out_path)
620             else:
621                 out_dir = args.output_dir
622                 out_path = os.path.join(out_dir, out_filename)
623                 out_name = out_filename
624

```

```

625         os.makedirs(out_dir, exist_ok=True)
626         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
627
628         # global_config isn't initialized this early; load the typed config so the trim
629         # clip carries the same configured stitching paddings + watermark as a real
630         # highlight (watermark default-on).
631         stitching = load_app_config().stitching
632         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
633                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
634         try:
635             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
636             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
637         except Exception as e:
638             print(f"[ERROR] Failed to compile trimmed segment: {e}")
639         return
640
641     if args.debug_clip is not None:
642         if not args.video_path:
643             print("[ERROR] Please provide --video-path to debug a specific clip.")
644             return
645         run_cli_diagnose_clip(args.video_path, args.debug_clip)
646         return
647
648     # Initialize Global Config and DB
649     global global_config, db_instance
650     global_config = load_app_config() # returns typed AppConfig
651     _enforce_startup_or_exit(global_config) # M5: fail fast on an incoherent ACTIVE
profile
652
653     # The CLI --output-dir overrides the configured output directory. It now lives
654     # on the typed model (OutputConfig.output_dir), so every consumer ? including
655     # /api/compile ? reads the same value instead of a write-only runtime hack.
656     global_config.output.output_dir = args.output_dir
657     os.makedirs(global_config.output.output_dir, exist_ok=True)
658
659     db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
660     db_instance = Database(db_path)
661
662     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
663     # the executor is in-process. Mark them failed so pollers see a terminal state.
664     swept = db_instance.fail_stale_jobs()
665     if swept:
666         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
667
668     # CLI processing mode (no web UI needed)
669     if args.video_path and not args.server:
670         print(f"[CLI] Processing video file: {args.video_path}")
671         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
672         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
673         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
674         storage_cfg = getattr(global_config, "storage", None)
675         try:
676             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
677         except ValueError as e:
678             print(f"[FAIL] {e}")
679         return
680         if input_ref.backend == "local" and not os.path.exists(input_ref.key):

```

```

681         print(f"[FAIL] Video file not found: {args.video_path}")
682         return
683     local_video = storage.acquire_local_input(args.video_path, storage_cfg)
684
685     video_id = compute_video_id(local_video)
686     was_reingest = db_instance.get_video(video_id) is not None
687
688     # Validate (with-context variant surfaces ffprobe_meta for the report)
689     print("[CLI] Validating...")
690     val_results, val_context = registry.run_all_with_context(local_video,
691 global_config)
692
693     failures = [r for r in val_results if not r.passed]
694
695     # Save video status
696     status = "failed" if failures else "processed"
697     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
698 in val_results])
699
700     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
701 "default_segmenter", "motion")
702     segmenter_actual = None
703     segmentation_ran = False
704     try:
705         print("\n" + "="*40)
706         print("VALIDATION SUMMARY")
707         print("="*40)
708         for r in val_results:
709             status_symbol = "[PASS]" if r.passed else "[FAIL]"
710             print(f"{status_symbol} {r.validator_name}: {r.message}")
711         print("="*40 + "\n")
712
713         if failures:
714             print("[FAIL] Video failed checks. Indexing halted.")
715             return
716
717         # Index
718         segmenter_cls = segmenter_registry.get(seg_name)
719         if not segmenter_cls:
720             print(f"[FAIL] Segmenter '{seg_name}' not found.")
721             return
722
723         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
724         segmenter_actual = seg_name
725         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
726         segmenter = segmenter_cls(db_instance, global_config)
727         result = segmenter.process_video(video_id, local_video,
728 max_frames=args.max_frames)
729         segmentation_ran = True
730
731         if isinstance(result, Failure):
732             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
733             print("[FAIL] Indexing halted for video.")
734             segments = []
735         else:
736             segments = result.value if hasattr(result, "value") else result
737             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
738 segments inside database: {db_path}")
739             # Destroy-AFTER-confirm: keep new on success, restore prior on
740 empty/failure.
741             _finalize_reingest(video_id, segments, play_wm, cand_wm)
742         finally:
743             # Always emit the per-video observability report. Never raises.
744             paths = emit_indexer_report(
745                 db=db_instance, video_id=video_id, video_path=args.video_path,
746                 config=global_config,

```

```

739         val_results=val_results, val_context=val_context,
740         segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
741         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
742     )
743     if paths:
744         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
745         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
746         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
747     return
748
749     # Start Local Server Mode
750     if args.server or not args.video_path:
751         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
752         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
753         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
754         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
755         _get_executor().submit(_drain_due_jobs)
756         uvicorn.run(app, host="127.0.0.1", port=args.port)
757
758     if __name__ == "__main__":
759         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
760         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
761         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
762         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
763         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
764         # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
765         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
766         import backend.main as _bm
767
768         _bm.main()
769

```

12. assistant (2026-06-21T14:36:59.568Z)

Excellent start. Now let me look at the compute backend and worker files:

13. user (2026-06-21T14:37:00.609Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\backend/api/job_worker.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/compute/base.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/compute/cloud_run.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/compute/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/worker/__init__.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/worker/__main__.py

```

14. assistant (2026-06-21T14:37:02.532Z)

Now let me read the key compute backend files:

15. user (2026-06-21T14:37:02.894Z)

```

1     """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2     for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4     Flow (the cloud-serving compute path):

```

```

5      1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6      --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8      completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9      it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10     cold-start is absorbed by the async poll.
11     3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12     ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14     The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17     The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19     owner-verified on the M7 real run.
20     ""
21
22     from __future__ import annotations
23
24     import json
25     import logging
26     import os
27     import tempfile
28     import uuid
29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
project: Optional[str] = None) -> str:
55

```

```

56         """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57         execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"
80
81
82     class CloudRunCompute(ComputeBackend):
83         """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85         def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                     project: Optional[str] = None, storage_config: Optional[dict] = None,
87                     policy: ExecutionPolicy = DEFAULT_POLICY,
88                     token: Optional[str] = None,
89                     container_overrides: Optional[tuple[str, ...]] = None) -> None:
90             self._client = run_client
91             self._job_name = job_name
92             self._region = region
93             self._project = project
94             self._storage_config = storage_config or {}
95             self._policy = policy
96             # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97             # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98             self._token = token
99             self._container_overrides = (
100                 _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101             )
102
103         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104             out_root = spec.out_uri.rstrip("/")
105             token = self._token or uuid.uuid4().hex
106             done_uri = f"{out_root}/_dispatch/{token}.done"
107             argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,

```

```

108         "--done-uri", done_uri]
109     if spec.segmenter:
110         argv += ["--segmenter", spec.segmenter]
111     for kv in (*spec.config_overrides, *self._container_overrides):
112         argv += ["--set", kv]
113
114     execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                     project=self._project)
116     LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117             self._job_name, execution, done_uri)
118
119     marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                        label="cloud-run-infer", logger=LOG)
121     video_id = str(marker.get("video_id", ""))
122     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124     rc = int(marker.get("returncode", 1))
125     if not video_id:
126         LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                     "video_id) ? treating as failure", self._job_name)
128         rc = rc or 1
129     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131     LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132     return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134     # --- storage-backed completion signal (bucket-poll) ---
135     def _read_marker(self, done_uri: str) -> Optional[dict]:
136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

16. user (2026-06-21T14:37:03.874Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2

```

```

3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9      are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14     (validator registry ? segmenter ? report), never a forked pipeline.
15     """
16     from __future__ import annotations
17
18     import argparse
19     import csv
20     import json
21     import logging
22     import os
23     import sys
24     import tempfile
25     from typing import Any, List
26
27     from backend import storage
28     from backend.compute import ComputeJobSpec, get_compute_backend
29     from backend.config import (
30         StartupCheckError,
31         enforce_startup_checks,
32         load_config,
33         probe_cuda_available,
34     )
35     from backend.infrastructure.database import Database
36     from backend.pipeline.segmenters.base import segmenter_registry
37     from backend.pipeline.validators.base import registry as validator_registry
38     from backend.reporting import emit_indexer_report
39     from backend.results import Failure
40     from backend.utils.hashing import compute_video_id
41
42     logger = logging.getLogger("backend.worker")
43
44
45     def _coerce(v: str) -> Any:
46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
str."""
47         low = v.lower()
48         if low in ("true", "false"):
49             return low == "true"
50         for cast in (int, float):
51             try:
52                 return cast(v)
53             except ValueError:
54                 pass
55         return v
56
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
60         for kv in sets or []:
61             key, sep, val = kv.partition("=")

```

```

62         if not sep:
63             raise ValueError(f"--set expects k=v, got {kv!r}")
64         parts = key.split(".")
65         obj = config
66         for p in parts[:-1]:
67             obj = getattr(obj, p)
68         setattr(obj, parts[-1], _coerce(val))
69
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
73         artifact (same schema as the rally-annotator labels)."""
74         with open(path, "w", newline="") as f:
75             w = csv.writer(f)
76             w.writerow(["start", "end", "shot_count", "ending_reason"])
77             for s in segments:
78                 w.writerow([
79                     f'{float(s.get("start_time", 0.0)):.3f}',
80                     f'{float(s.get("end_time", 0.0)):.3f}',
81                     s.get("shot_count", ""),
82                     s.get("ending_reason", s.get("shot_count_confidence", "")),
83                 ])
84
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
87         with open(path, "w") as f:
88             json.dump({
89                 "video_id": video_id,
90                 "status": status,
91                 "segment_count": len(segments),
92                 "segments": [{"start": float(s.get("start_time", 0.0)),
93                             "end": float(s.get("end_time", 0.0))} for s in segments],
94             }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc

```

```

once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135             logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                         result.video_id, result.returncode, result.outputs_uri)
137         return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                           status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
144         try:
145             marker = {"video_id": video_id, "returncode": returncode,
146                      "segment_count": segment_count, "status": status}
147             local = os.path.join(scratch, "_done.json")
148             with open(local, "w", encoding="utf-8") as f:
149                 json.dump(marker, f)
150             storage.put(local, done_uri, config=storage_cfg)
151             logger.info("[worker] wrote completion marker -> %s", done_uri)
152         except Exception as e: # never fail a good run because the marker push hiccuped
153             logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154
155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
163         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
164         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
165         # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
166         compute = get_compute_backend(config)
167         if compute is not None:
168             return _dispatch_remote(compute, args)

```

```

169
170     storage_cfg = config.storage.model_dump()
171
172     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
173     os.makedirs(scratch, exist_ok=True)
174     config.output.output_dir = scratch
175
176     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
177     local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
config=storage_cfg)
178     print(f"[worker] pulled {args.video_uri} -> {local_video}")
179
180     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
181     video_id = compute_video_id(local_video)
182
183     # 3. VALIDATE (same registry as the main CLI).
184     val_results, val_context = validator_registry.run_all_with_context(local_video,
config)
185     failures = [r for r in val_results if not r.passed]
186     db = Database(os.path.join(scratch, config.output.db_name))
187     db.add_video(video_id, local_video, "failed" if failures else "processed",
[r.model_dump() for r in val_results])
188
189
190     segments: List[dict] = []
191     segmenter_actual = None
192     segmentation_ran = False
193     status = "failed"
194     try:
195         if failures:
196             print("[worker] validation FAILED: "
+ "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
197             return 2
198         seg_name = args.segmenter or config.indexing.default_segmenter
199         segmenter_cls = segmenter_registry.get(seg_name)
200         if not segmenter_cls:
201             print(f"[worker] unknown segmenter: {seg_name!r} "
f"(have {list(segmenter_registry.list_available())})")
202             return 2
203         segmenter_actual = seg_name
204         result = segmenter_cls(db, config).process_video(video_id, local_video,
max_frames=args.max_frames)
205         segmentation_ran = True
206         if isinstance(result, Failure):
207             print(f"[worker] segmenter FAILED: {result.message}")
208             return 3
209         segments = result.value if hasattr(result, "value") else result
210         status = "success"
211         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
212         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
213         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
214         if not segments:
215             detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
or getattr(config.indexing, "detector_impl", None) or
"auto")
216             logger.warning(
217                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
218                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
219                 "logs above; common causes: native runner UNAVAILABLE

```

```

(weights/repo/WASB_PYTHON env), "
223         "the WASB env produced no detections, or the input resolution differs
from the tuned "
224         "proxy resolution. Re-run with -v for the native command + frame
counts.",
225         video_id, segmenter_actual, detector_impl,
226     )
227     finally:
228         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229         emit_indexer_report(
230             db=db, video_id=video_id, video_path=local_video, config=config,
231             val_results=val_results, val_context=val_context,
232             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233             was_reingest=False, segmentation_ran=segmentation_ran,
234         )
235
236     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
237     out_local = os.path.join(scratch, "outputs", video_id)
238     os.makedirs(out_local, exist_ok=True)
239     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
240     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
241
242     out_prefix = args.out_uri.rstrip("/")
243     pushed = []
244     for fn in sorted(os.listdir(out_local)):
245         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
246         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
247         pushed.append(uri)
248
249     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
250     for u in pushed:
251         print("    ", u)
252
253     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
254     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
255     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
256     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
257     if getattr(args, "done_uri", None):
258         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
259     return 0
260
261
262     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
263     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
264     _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
265
266
267     def _probe_video_fps_width(path: str):
268         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
269
270         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
271         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
272         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for

```

```

the
273         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
274         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
275         rather than guesses.
276         """
277         import json
278         import subprocess
279         try:
280             out = subprocess.check_output(
281                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
282                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
283                 stderr=subprocess.DEVNULL).decode()
284             st = (json.loads(out).get("streams") or [{}])[0]
285             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
286             fps = float(num) / float(den or "1")
287             width = float(st.get("width") or 0)
288             if fps > 0 and width > 0:
289                 return fps, width
290         except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
291             pass
292         return None
293
294
295     def _resolve_train_detector(args: argparse.Namespace, config: Any):
296         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
297         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
298         stub=CPU mock). Returns the runner, or prints an error + returns None.
299
300         Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build
the
301         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
302         no shuttle detector and is rejected.
303         """
304         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
305
306         seg_cls = segmenter_registry.get(args.segmenter)
307         if not seg_cls:
308             print(f"[worker] unknown segmenter: {args.segmenter!r} "
309                   f"(have {list(segmenter_registry.list_available())})")
310             return None
311         seg = seg_cls(None, config)
312         if not isinstance(seg, TrajectoryHybridSegmenter):
313             print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
314                   f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid /
fusion_hybrid).")
315             return None
316         try:
317             runner, _ = seg._resolve_runner(config.get("indexing", {}))
318         except NotImplementedError as e:
319             # e.g. detector_impl='native' for a family without a native runner (tracknet, or
320             # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
321             print(f"[worker] detector not available: {e}")
322             return None
323         ok, msg = runner.healthcheck()
324         if not ok:
325             print(f"[worker] detector environment not ready: {msg}")
326             return None
327         print(f"[worker] detector ready ({args.segmenter}): {msg}")
328         return runner

```

```

329
330
331     def cmd_train(args: argparse.Namespace) -> int:
332         """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
333         to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
334
335         Two trajectory sources (the train pipeline + harness are identical either way):
336         * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
337             shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
338         * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
339         DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
340             is the M3.5 "first real training" path; only the trajectory source flips.
341         """
342         import subprocess
343
344         config = load_config(args.config) if args.config else load_config()
345         _apply_overrides(config, args.set)
346         if not _run_startup_checks(config):
347             return 2
348         storage_cfg = config.storage.model_dump()
349
350         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
351         labels_dir = os.path.join(scratch, "labels")
352         traj_dir = os.path.join(scratch, "trajectories")
353         videos_dir = os.path.join(scratch, "videos")
354         os.makedirs(labels_dir, exist_ok=True)
355         os.makedirs(traj_dir, exist_ok=True)
356
357         corpus = args.corpus_uri.rstrip("/")
358         label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
359                             if u.lower().endswith(".csv"))
360         if len(label_uris) < 2:
361             print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
362                   f"under {corpus}/labels/")
363             return 2
364
365         # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
366         runner = None
367         video_by_stem: dict = {}
368         if args.trajectory_source == "detector":
369             os.makedirs(videos_dir, exist_ok=True)
370             runner = _resolve_train_detector(args, config)
371             if runner is None:
372                 return 2
373             for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
374                 vname = storage.parse_uri(vuri).name
375                 if not vname.lower().endswith(_VIDEO_EXTS):
376                     continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
377                 video_by_stem[os.path.splitext(vname)[0]] = vuri
378
379         print(f"[worker] trajectory source: {args.trajectory_source}")
380         specs: List[dict] = []
381         for uri in label_uris:
382             fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
383             name = fname.replace(".rallies.csv", "").replace(".csv", "")

```

```

384         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
385         if args.trajectory_source == "detector":
386             vuri = video_by_stem.get(name)
387             if not vuri:
388                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
389                 continue
390             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
391                                     config=storage_cfg)
392             video_id = compute_video_id(local_video)
393             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
394             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
395             meta = _probe_video_fps_width(local_video)
396             if meta is None:
397                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
398                 continue
399             fps, frame_width = meta
400             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
401             if not traj:
402                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
403                 continue
404             specs.append({"name": name, "trajectory": traj, "labels": local_label,
405                          "fps": fps, "frame_width": frame_width})
406         else:
407             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
408             traj = os.path.join(traj_dir, f"{name}_traj.csv")
409             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
410             specs.append({"name": name, "trajectory": traj, "labels": local_label,
411                          "fps": args.fps, "frame_width": args.frame_width})
412
413         if len(specs) < 2:
414             print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
415             return 2
416         manifest_path = os.path.join(scratch, "manifest.json")
417         with open(manifest_path, "w", encoding="utf-8") as f:
418             json.dump(specs, f, indent=2)
419         src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
420         print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
421
422         out_dir = os.path.join(scratch, "artifacts")
423         cmd = [sys.executable, "-m", "training.gen0.harness",
424               "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
425         if args.floor:
426             floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
427                           if "://" in args.floor else args.floor)
428             cmd += ["--floor", floor_local]
429         print("[worker] training:", " ".join(cmd))
430         rc = subprocess.run(cmd).returncode
431         if rc != 0:
432             print(f"[worker] gen0 harness failed (rc={rc})")
433             return rc
434
435         # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
436         bundle = os.path.join(out_dir, args.version)

```

```

437     out_prefix = args.out_uri.rstrip("/")
438     pushed = []
439     for fn in sorted(os.listdir(bundle)):
440         uri = f"{out_prefix}/weights/{args.version}/{fn}"
441         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
442         pushed.append(uri)
443     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
444     for u in pushed:
445         print("    ", u)
446     return 0
447
448
449     def build_parser() -> argparse.ArgumentParser:
450         p = argparse.ArgumentParser(prog="python -m backend.worker",
451                                     description="Storage-aware worker: pull ? run pipeline ?
push.")
452         p.add_argument("-v", "--verbose", action="store_true",
453                       help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
454         sub = p.add_subparsers(dest="cmd", required=True)
455
456         pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
457         pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
458         pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
459         pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
460         pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
461         pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
462         pi.add_argument("--max-frames", type=int, default=None)
463         pi.add_argument("--done-uri", default=None,
464                       help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
465                       "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
466                       "Default-OFF: omit it for the normal local run.")
467         pi.add_argument("--set", action="append", default=[], metavar="k=v",
468                       help="config override, e.g. --set indexing.skip_ai_handoff=true")
469         pi.set_defaults(func=cmd_infer)
470
471         pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")
472         pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
473                       "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
474         pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
475         pt.add_argument("--version", required=True, help="semver for the artifact bundle")
476         pt.add_argument("--trajectory-source", choices=["synth", "detector"],
477                       default="synth",
478                       help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
479                       "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
480                       "(needs videos/ in the corpus; honours detector_impl).")
481         pt.add_argument("--segmenter", default="wasb_hybrid",
482                       help="trajectory hybrid whose detector is used for --trajectory-
source detector")
483         pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
484         pt.add_argument("--config", default=None)
485         pt.add_argument("--scratch", default=None)

```

```

485     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
486     pt.add_argument("--frame-width", type=float, default=1280.0,
487                     help="synth trajectory frame width (detector probes the video)")
488     pt.add_argument("--set", action="append", default=[], metavar="k=v")
489     pt.set_defaults(func=cmd_train)
490     return p
491
492
493     def _setup_logging(verbose: bool) -> None:
494         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
495         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
496         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
497         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
498         entry), not at import, so importing the module never reconfigures a caller's
logging."""
499         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
500         # configured logging at import), which would leave the level at WARNING and re-
suppress the
501         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
502         logging.basicConfig(
503             level=logging.DEBUG if verbose else logging.INFO,
504             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
505             datefmt="%H:%M:%S",
506             force=True,
507         )
508
509
510     def main(argv=None) -> int:
511         args = build_parser().parse_args(argv)
512         _setup_logging(getattr(args, "verbose", False))
513         return args.func(args)
514
515
516     if __name__ == "__main__":
517         sys.exit(main())
518

```

17. assistant (2026-06-21T14:37:05.890Z)

Now let me check the database implementation to understand job state storage:

18. user (2026-06-21T14:37:07.104Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\backend\infrastructure\database.py:class Database:

19. user (2026-06-21T14:37:09.507Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     from backend import billing # pure cost math (no backend imports ? no cycle)
8
9     logger = logging.getLogger(__name__)

```

```

10
11 class Database:
12     def __init__(self, db_path: str):
13         self.db_path = db_path
14         self._init_db()
15
16     @contextmanager
17     def _connect(self):
18         """Transaction scope that also CLOSES the connection.
19
20         ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
21         never closes, so every call leaked its connection to GC (lingering file
22         handles on the WAL sidecars under Windows). Internal methods use this.
23         """
24         conn = self._get_connection()
25         try:
26             with conn:
27                 yield conn
28         finally:
29             conn.close()
30
31     def _get_connection(self):
32         # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
33         # adds concurrent writers (a swallowed 'database is locked' silently drops
34         segments).
35         conn = sqlite3.connect(self.db_path, timeout=30.0)
36         conn.row_factory = sqlite3.Row
37         # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
38         conn.execute("PRAGMA foreign_keys = ON")
39         # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
40         conn.execute("PRAGMA busy_timeout = 30000")
41         try:
42             conn.execute("PRAGMA journal_mode = WAL")
43         except sqlite3.OperationalError:
44             pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
45         return conn
46
47     def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
48         """Run ONE bool-returning single-statement write inside a connection scope.
49
50         Captures the connect/commit/except-logger.error/return-False boilerplate shared
51         by
52
53         the bool-returning single-statement writers. ``error_msg`` is the caller's exact
54         per-method message (already formatted with its own ids) ? logged verbatim on a
55         swallowed write fault so each writer keeps its specific log text. Returns True
56         on
57
58         success, False (after logging ``error_msg``) on any exception."""
59         try:
60             with self._connect() as conn:
61                 conn.cursor().execute(sql, params)
62                 conn.commit()
63             return True
64         except Exception as e:
65             logger.error(f"{error_msg}: {e}")
66             return False
67
68     @staticmethod
69     def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
70         """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.
71
72         A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn``
73         transaction, so an interrupted v3/v4 block can leave the column added while
74         ``user_version`` stays un-bumped ? and the next open would re-run the same
75         ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
76         future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
```

```

Swallow
72         ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
73         ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
74         """
75         try:
76             cursor.execute(ddl)
77         except sqlite3.OperationalError as exc:
78             if "duplicate column name" not in str(exc).lower():
79                 raise
80
81     def _init_db(self):
82         """Initializes tables for videos, play_segments, and events."""
83         with self._connect() as conn:
84             cursor = conn.cursor()
85
86             self._create_base_tables(cursor)
87
88             # Versioned migrations live here. PRAGMA user_version is the schema
89             # contract ? bump it for every change and add an ordered `if version < N`
90             # block below. Tests assert the expected version, so accidental drift fails
91             version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93             if version < 1:
94                 self._migrate_to_v1(cursor)
95
96             if version < 2:
97                 self._migrate_to_v2(cursor)
98
99             if version < 3:
100                 self._migrate_to_v3(cursor)

```

20. assistant (2026-06-21T14:37:11.577Z)

Let me continue reading the database file and look for job-related tables:

21. user (2026-06-21T14:37:12.617Z)

```

100             self._migrate_to_v3(cursor)
101
102             if version < 4:
103                 self._migrate_to_v4(cursor)
104
105             if version < 5:
106                 self._migrate_to_v5(cursor)
107
108             if version < 6:
109                 self._migrate_to_v6(cursor)
110             # future: if version < 7: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 7
111
112             conn.commit()
113
114     def _create_base_tables(self, cursor):
115         """Create the videos / play_segments / events base tables (idempotent)."""
116         # Videos Table
117         cursor.execute("""
118             CREATE TABLE IF NOT EXISTS videos (
119                 id TEXT PRIMARY KEY,
120                 filepath TEXT NOT NULL,
121                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
122                 status TEXT NOT NULL,
123                 validation_results TEXT
124             )

```

```

125         """)
126
127     # Play Segments Table (Extensible metadata JSON)
128     cursor.execute("""
129         CREATE TABLE IF NOT EXISTS play_segments (
130             id INTEGER PRIMARY KEY AUTOINCREMENT,
131             video_id TEXT NOT NULL,
132             start_time REAL NOT NULL,
133             end_time REAL NOT NULL,
134             duration REAL NOT NULL,
135             server TEXT,
136             receiver TEXT,
137             metadata TEXT,
138             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
139         )
140     """)
141
142     # Events Table
143     cursor.execute("""
144         CREATE TABLE IF NOT EXISTS events (
145             id INTEGER PRIMARY KEY AUTOINCREMENT,
146             segment_id INTEGER NOT NULL,
147             timestamp REAL NOT NULL,
148             type TEXT NOT NULL,
149             player TEXT,
150             details TEXT,
151             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
152         )
153     """)
154
155     def _migrate_to_v1(self, cursor):
156         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
157         # Common fields are columns; detector-specific metrics go in `stats` JSON,
158         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
159         cursor.execute("""
160             CREATE TABLE IF NOT EXISTS candidate_segments (
161                 id INTEGER PRIMARY KEY AUTOINCREMENT,
162                 video_id TEXT NOT NULL,
163                 detector TEXT NOT NULL,
164                 chunk_index INTEGER,
165                 start_time REAL NOT NULL,
166                 end_time REAL NOT NULL,
167                 duration REAL NOT NULL,
168                 confidence REAL,
169                 stats TEXT,
170                 detection_params TEXT,
171                 confirmed_segment_id INTEGER,
172                 human_label TEXT,
173                 notes TEXT,
174                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
175                 FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
176                 FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
177             DELETE SET NULL
178         )
179     """)
180     cursor.execute("""
181         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
182         ON candidate_segments(video_id, detector)
183     """)
184     cursor.execute("PRAGMA user_version = 1")
185
186     def _migrate_to_v2(self, cursor):
187         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
188         # /api/process request. `status` is the EXECUTION state of the job
189         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video

```

```

189         # that failed validation) lives inside `result` JSON as result["status"].
190         # `result` is the full sync-era response dict (incl. report paths), so the
191         # observability report is the job's artifact, per the plan.
192         cursor.execute("""
193             CREATE TABLE IF NOT EXISTS jobs (
194                 id TEXT PRIMARY KEY,
195                 video_path TEXT NOT NULL,
196                 video_id TEXT,
197                 segmenter TEXT,
198                 player_pool TEXT,
199                 max_frames INTEGER,
200                 status TEXT NOT NULL DEFAULT 'queued',
201                 progress TEXT,
202                 error TEXT,
203                 result TEXT,
204                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
205                 started_at TIMESTAMP,
206                 finished_at TIMESTAMP
207             )
208         """)
209         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
210         cursor.execute("PRAGMA user_version = 2")
211
212     def _migrate_to_v3(self, cursor):
213         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
214         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
215         # due
216         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
217         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
218         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
219         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
220         # them.
221         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
222         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
223         TIMESTAMP")
224         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
225         next_poll_at)")
226         cursor.execute("PRAGMA user_version = 3")
227
228     def _migrate_to_v4(self, cursor):
229         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
230         # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
231         # path can scope rows to an owner. **NULL = local install = byte-identical to
232         # today** ? every existing row stays NULL and every existing query (which never
233         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
234         # the ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
235         # them.
236         # Additive ONLY: no served-behavior change rides on this slice.
237         self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
238         TEXT")
239         self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
240         user_id TEXT")
241         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
242         # of index churn while making the eventual per-user lookups cheap.
243         cursor.execute(
244             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
245             "WHERE user_id IS NOT NULL")
246         cursor.execute(
247             "CREATE INDEX IF NOT EXISTS idx_candidates_user "
248             "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
249         cursor.execute("PRAGMA user_version = 4")
250
251     def _migrate_to_v5(self, cursor):
252         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
253         # version): the resolved per-user `fusion_config` overlay + an INLINE

```

```

247         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
248         # and provenance that earned it. `is_active` pins the one row the serving bridge
249         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice

```

22. assistant (2026-06-21T14:37:14.708Z)

Now let me check PORT handling and health check in main.py and the gcp README:

23. user (2026-06-21T14:37:15.478Z)

```

1         # GCP provisioning runbook ? DECOUPLED_COMPUTE GPU + storage (asia-south1)
2
3         > **? GCP is the SOLE compute stack (ADR-013, 2026-06-20).** DigitalOcean is **dropped
for compute**
4         > (DO GPU Droplets aren't enabled on the account + aren't cleanly credit-funded ? DO
charges newer
5         > customers' cards for GPU; Paperspace is subscription-gated). **All training + serving
runs on GCP.**
6         > The `deploy/digitalocean/` scripts are kept only as **provider-agnostic** Linux-GPU
setup that this
7         > runbook reuses. ? Capacity note: asia-south1 (Mumbai) GPUs are frequently stocked-out
? sweep zones /
8         > fall back to asia-southeast1 (Singapore) etc.; weights still land in the asia-south1
bucket.
9
10        Co-located **GPU VM + GCS bucket in ONE region** (asia-south1 / Mumbai) for the first
real
11        (non-mock) WASB training + inference. This is the **DO?GCP pivot** (ADR-010):
DigitalOcean GPU
12        Droplets are North-America-only (no Singapore/India GPU), so we moved GPU + storage to
GCP, which
13        has GPUs in asia-south1 (Mumbai) and asia-southeast1 (Singapore). Owner picked
**Mumbai** (closest
14        to Bangalore ? lowest latency).
15
16        > **Cost discipline (read first):** a running GPU VM is the only meaningful cost. Use
the cheapest
17        > adequate GPU (**T4**), **STOP or DELETE the VM the moment a run finishes** (a stopped
VM bills only
18        > for its disk), and keep the **$100 budget alert** live. `teardown.sh` switches
everything off.
19
20        ---
21
22        ## 0. Live resources (created 2026-06-20 ? the "switch-off" inventory)
23
24        | Resource | Identifier | Cost when idle | How to switch off |
25        |---|---|---|---|
26        | Project | `gen-lang-client-0306026826` (billing **Khelsutra Billing**
`01420D-419EE0-53D83C`) | ? | n/a (shared Gemini project) |
27        | GCS bucket | `gs://khelsutra-rally-corpus` (asia-south1, UBLA) | ~$0.02/GB/mo storage
| `gcloud storage rm -r` (? data loss) |
28        | Service account | `rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com`
(`roles/storage.objectAdmin`) | $0 | `gcloud iam service-accounts delete` |
29        | SA key | `~/gcp/rally-worker-sa.json` (local only, **never** committed) | $0 | delete
the file + the key |
30        | Budget alert | `rally-100usd-guard` ? $100, alerts at 50/90/100% | $0 | `gcloud
billing budgets delete` |
31        | **GPU VM** | **NOT created** ? blocked on `GPUS_ALL_REGIONS` quota (see $2) |
**~$0.35/hr (T4) running, ~$0.01/hr stopped** | `teardown.sh` (stop or delete) |
32        | APIs enabled | serviceusage, compute, cloudbilling, billingbudgets, iam,
cloudresourcemanager | $0 | leave on (free) |
33

```

```

34     Run `bash deploy/gcp/teardown.sh` (see §6) to stop/delete the billable bits between
sessions.
35
36     ---
37
38     ## 1. One-time project bootstrap (owner, console ? already done 2026-06-20)
39
40     A bare Gemini/AI-Studio project has the **Service Usage API disabled**, which blocks
*all* gcloud
41     provisioning (gcloud's own `services enable` needs that API ? chicken-and-egg). These
were enabled
42     in the console; recorded here so a fresh project can be bootstrapped the same way:
43
44     1. **Service Usage API** ?
https://console.developers.google.com/apis/api/serviceusage.googleapis.com/overview?project=gen-
lang-client-0306026826
45     2. **Link a billing account** (a payment method) ?
https://console.cloud.google.com/billing/linkedaccount?project=gen-lang-client-0306026826
46     3. After (1), the rest enable via gcloud (done): `gcloud services enable
compute.googleapis.com cloudbilling.googleapis.com billingbudgets.googleapis.com
iam.googleapis.com cloudresourcemanager.googleapis.com`
47
48     `provision.sh` (§5) is idempotent and re-runs all of step (3) + the bucket/SA/budget.
49
50     ---
51
52     ## 2. ? GPU quota ? the one remaining owner action
53
54     New GCP projects start at **`GPUS_ALL_REGIONS` = 0**, the *global* cap that gates GPU
creation even
55     when a regional per-type quota is non-zero. Confirmed 2026-06-20:
56
57     ```
58     asia-south1  NVIDIA_T4_GPUS = 1    NVIDIA_L4_GPUS = 1    (regional: OK)
59     GLOBAL      GPUS_ALL_REGIONS = 0    ? BLOCKS the VM
60     ```
61
62     **Owner:** IAM & Admin ? **Quotas** ? filter **"GPUs (all regions)"** ? Edit ? request
**1** ? submit.
63     (For a single T4 this is usually approved in minutes; sometimes a short review.) Re-
check:
64
65     ```bash
66     gcloud compute project-info describe --project gen-lang-client-0306026826 \
67     --flatten="quotas[]" --format="table(quotas.metric,quotas.limit)" | grep
GPUS_ALL_REGIONS
68     ```
69
70     When that reads `1`, run §3.
71
72     > **? Spot does NOT skip the quota.** TESTED 2026-06-20: a **Spot** T4 create also fails
with
73     > `Quota 'GPUS_ALL_REGIONS' exceeded. Limit: 0.0 globally` ? `GPUS_ALL_REGIONS` gates
**both**
74     > on-demand AND preemptible GPUs, so the increase above is required either way. (A
pending request
75     > was filed via the Cloud Quotas API: `POST ?/quotaPreferences` `GPUS-ALL-REGIONS-per-
project`
76     > ? 1.) **Note (2026-06-20, ADR-013 supersedes the ADR-011 split): GCP is the SOLE
compute stack ?
77     > BOTH training and serving run on GCP; DigitalOcean is dropped for compute.** Spot is
still cheaper
78     > once the quota is granted, and the resumable cache makes preemption recoverable.
79
80     ---

```

```

81
82     ## 3. Create the GPU VM (after quota ? 1)
83
84     > **? Recommended: `bash deploy/gcp/create_gpu_vm.sh`** ? the one-command way. It sweeps
zones for
85     > L4 capacity (Mumbai ? Singapore), uses the driver-ready image, attaches the `rally-
worker` SA, and
86     > **always bakes in the Ops Agent** (`--metadata-from-file startup-
script=deploy/gcp/ops-agent-startup.sh`)
87     > so Observability/GPU graphs populate automatically (no more "Ops Agent missing"). It
prints the
88     > zone + the SSH/DELETE commands. `RALLY_GPU=t4` for a T4; `RALLY_VM_NAME=?` to rename.
The manual
89     > command below is kept for reference / customization.
90
91     T4 is plenty for WASB + Gen-0. A local SSD gives a fast `/scratch`. *(The native runner
defaults to
92     **streaming** ? it decodes frames in-process, skipping the slow cv2 PNG extraction; bit-
identical to disk,
93     verified on a real GPU 2026-06-20. Set `indexing.wasb.stream_video=false` only if a
container/codecs cv2
94     can't seek; `WASB_SCRATCH` still backs the disk path.)*
95
96     ```bash
97     gcloud compute instances create rally-gpu \
98         --project=gen-lang-client-0306026826 \
99         --zone=asia-south1-a \
100        --machine-type=n1-standard-8 \
101        --accelerator=type=nvidia-tesla-t4,count=1 \
102        --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
103        --image-family=ubuntu-2404-lts-amd64 --image-project=ubuntu-os-cloud \
104        --boot-disk-size=120GB --boot-disk-type=pd-balanced \
105        --local-ssd=interface=NVME \
106        --service-account=rally-worker@gen-lang-client-0306026826.iam.gserviceaccount.com \
107        --scopes=cloud-platform
108    ```
109
110    **Spot variant (cheaper; still needs the $2 quota ? Spot is NOT a quota bypass):** same
command with
111    `--provisioning-model=SPOT` (replacing `STANDARD`). Add `--instance-termination-
action=STOP` to keep
112    the disk on preemption so a re-run resumes. Cheaper, can be preempted.
113
114    > **Validated image (2026-06-20):** `--image-family=common-cu129-ubuntu-2204-nvidia-580
115    > --image-project=deeplearning-platform-release` ships the NVIDIA driver (no install-on-
boot wait;
116    > `nvidia-smi` works immediately) ? used for the first real L4 run. `gpu_setup.sh`
installs miniconda
117    > on it. **T4 was capacity-stocked-out in `asia-south1-a`; an L4 (`g2-standard-4`, no
`--accelerator`
118    > flag ? L4 is built into g2) in `asia-south1-c` landed** ? try L4/other zones if T4 is
unavailable.
119
120    ### 3b. Observability ? Ops Agent (GPU graphs + logs in the console)
121    The VM **Observability ? GPU** graphs + Logs need the **Ops Agent** (the console's OS-
policy install is
122    blocked on DLVM images). Install it directly + grant the worker SA the write roles (it
otherwise only has
123    storage), then GPU/NVML metrics + syslog flow to Cloud Monitoring/Logging (tab populates
in ~2-3 min):
124    ```bash
125    P=gen-lang-client-0306026826; SA=rally-worker@$P.iam.gserviceaccount.com
126    # 1. ENABLE the APIs (bare Gemini project has them OFF ? the agent installs but its
exports get
127    #     PermissionDenied/SERVICE_DISABLED until these are on; THIS was the empty-graphs

```

```

cause 2026-06-20):
128     gcloud services enable logging.googleapis.com monitoring.googleapis.com --project $P
129     # 2. let rally-worker publish metrics + logs:
130     gcloud projects add-iam-policy-binding $P --member="serviceAccount:$SA"
131     --role=roles/monitoring.metricWriter --condition=None
132     gcloud projects add-iam-policy-binding $P --member="serviceAccount:$SA"
133     --role=roles/logging.logWriter --condition=None
134     # 3. on the box (created with --scopes=cloud-platform):
135     curl -sSO https://dl.google.com/cloudagents/add-google-cloud-ops-agent-repo.sh && sudo
136     bash add-google-cloud-ops-agent-repo.sh --also-install
137     # (if the agent was installed BEFORE the APIs were enabled: `sudo systemctl restart
138     google-cloud-ops-agent`)
139     ```
140     > Best: bake `curl ?add-google-cloud-ops-agent-repo.sh|bash --also-install` into the VM
141     > `--metadata=startup-script` so observability is automatic on every box.
142     > **Long remote steps must be detached.** A plain `ssh ? "<long cmd>"` dies if the SSH
143     drops (seen
144     > mid-build, 2026-06-20) ? run them `nohup`'d on the box (`nohup bash setup.sh >log 2>&1
145     &`) and poll
146     > the log, so a dropped connection can't kill the install.
147     > If `asia-south1-a` reports no T4 capacity, try `-b` / `-c`. The Ubuntu image needs the
148     NVIDIA
149     > driver: either add `--metadata=install-nvidia-driver=True` style startup, or after SSH
150     run GCP's
151     > driver installer, **or** use a Deep-Learning-VM image (`--image-family=common-
152     cu124-ubuntu-2204`
153     > `--image-project=deeplearning-platform-release`) which ships the driver. Confirm
154     `nvidia-smi` works
155     > before §4.
156     SSH: `gcloud compute ssh rally-gpu --zone=asia-south1-a`.
157     ---
158     ## 4. Port the repo + WASB env onto the box
159     The `deploy/digitalocean/` scripts are **provider-agnostic** (they detect the local NVMe
160     / install
161     torch); reuse them on GCP:
162     ```bash
163     # from your machine ? ship the repo, BAKING the source commit so the trained bundle is
164     # provenance-stamped. A git-archive tarball carries no .git, so training/gen0/harness.py
165     would
166     # otherwise record git_sha:"unknown" (seen on the 2026-06-20 L4 run). It reads, in
167     order:
168     # live `git rev-parse` ? $RALLY_GIT_SHA ? a GIT_SHA file at the repo root ? "unknown".
169     git rev-parse HEAD > /tmp/GIT_SHA
170     git archive --format=tar.gz --add-file=/tmp/GIT_SHA -o /tmp/rally.tgz HEAD # GIT_SHA ?
171     archive root (git ?2.38)
172     gcloud compute scp /tmp/rally.tgz rally-gpu:/root/ --zone=asia-south1-a
173     gcloud compute ssh rally-gpu --zone=asia-south1-a -- 'mkdir -p ~/rally && tar -xzf
174     ~/rally.tgz -C ~/rally'
175     # (Alternative, if not baking the file: `export RALLY_GIT_SHA=$(git rev-parse HEAD)` on
176     the box
177     # in the same shell that runs `python -m backend.worker train ?`.)
178     # on the box:
179     cd ~/rally
180     sudo bash deploy/digitalocean/mount_scratch.sh && export TMPDIR=/scratch # detects the
181     local NVMe
182     ./deploy/digitalocean/bootstrap.sh --gpu # venv +
183     torch cu124 (confirm cuda True)

```

```

174     . .venv/bin/activate && pip install -e .[storage-gcs]           # GCS backend
175     bash deploy/digitalocean/gpu_setup.sh                          # native
`wasb` conda env + WASB-SBDT
176
177     # secrets (NOT in repo): the GCS SA JSON + a rotated Gemini key
178     gcloud compute scp ~/.gcp/rally-worker-sa.json rally-gpu:/root/.gcp/sa.json --zone=asia-
south1-a
179     # on the box: export GOOGLE_APPLICATION_CREDENTIALS=/root/.gcp/sa.json
180     mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY
&& chmod 600 /root/.keys/GEMINI_API_KEY
181     ```
182
183     WASB weights (`wasb_badminton_best.pth.tar`) must be present at
184     `~/models/WASB-SBDT/pretrained_weights/` (gpu_setup.sh does NOT fetch them ? stage from
Drive/bucket).
185     Point the native runner at them: `export WASB_WEIGHTS=~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar`
186     and `export WASB_PYTHON=$(conda run -n wasb which python)` (or the env's python path).
See
187     `docs/TRACKNET_WSL_SETUP.md` for the WASB env. **WASB-C3:** weight provenance is an open
legal item
188     before sharing any trajectories/model externally ? internal extraction + Gen-0 is fine.
189
190     ---
191
192     ## 5. Stage the corpus + run the first REAL train/infer
193
194     Config for GCS (no secrets in config ? auth via `GOOGLE_APPLICATION_CREDENTIALS`):
195
196     ```json
197     { "sport": "badminton",
198       "indexing": { "default_segmenter": "wasb_hybrid", "detector_impl": "native" },
199       "storage": { "backend": "local", "gcs": { "project": "gen-lang-client-0306026826" } }
200     }
201     ```
202
203     ```bash
204     # stage the 7 Done videos + 7 labels into the bucket (from the co-located box ? fast):
205     #   videos -> gs://khelsutra-rally-corpus/videos/   labels -> gs://khelsutra-rally-
corpus/labels/
206     # (owner handles the actual copy; see "Upload options" below)
207
208     # FIRST REAL training (real WASB trajectories, native runner):
209     RALLY_DETECTOR_IMPL=native python -m backend.worker train \
210       --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-rally-corpus \
211       --version 0.1.0 --trajectory-source detector --segmenter wasb_hybrid --config
config.json
212
213     # FIRST REAL inference on the held-out Video 1 (GX010135_proxy.mp4, no labels):
214     python -m backend.worker infer \
215       --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
216       --out-uri gcs://khelsutra-rally-corpus --segmenter wasb_hybrid --config config.json
217
218     ### Upload options (how to get a video in + run inference)
219     The worker resolves `--video-uri` via the storage layer ? pick whichever fits:
220     - **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-
corpus/videos/clip.mp4`. Upload with
221       `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
222     - **S3 / D0 Spaces / R2:** `s3://bucket/clip.mp4` (install `[storage-s3]`, creds in
`~/aws/credentials`).
223     - **Local upload:** scp the file to the box and pass a bare path /
`local:///root/clip.mp4` (identity passthrough, no copy).
224     - **Google Drive (assumes Drive is MOUNTED):** mount Google Drive for Desktop ? a fair
requirement to

```

```

225     use Drive files locally ? and reference them with `gdrive://<path-under-My-Drive>`.
The backend resolves
226     it against your local mount (auto-detected: Windows `G:\My Drive`, macOS
227     `~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`; override with
`GDRIVE_MOUNT_ROOT` or
228     `storage.gdrive.mount_root`), then it's plain local I/O ? **no service account, no
API, no extra install.**
229     E.g. the golden-corpus folder `~/drive/folders/1sHGL7iacnmM80ChT-p2cKy638gK504T_`
mounts at
230     `G:\My Drive\Sharing\Annotation Videos\Golden Label Videos Request - June 15\`, so
231     `gdrive://Sharing/Annotation Videos/Golden Label Videos Request - June 15/[Done] Video
4 - Badminton/GX020094_proxy.mp4`
232     (or just the raw `G:\My Drive\?` path ? the `local` backend handles that too). ? A
**headless GPU VM has
233     no mount** ? there, stage Drive?bucket and read `gcs://`, or `gdown <file-id> -O
clip.mp4` to a local path.
234
235     ---
236
237     ## 6. Switch off between sessions (cost control)
238
239     > **? DELETE, don't just STOP.** A *stopped* VM halts compute billing but **still bills
for its
240     > persistent disk** (pd-ssd ? $0.17/GB/mo ? a 200 GB boot disk ? **$34/mo**). Only
**delete** reaches
241     > $0. So `teardown.sh` **defaults to `delete`** (owner cost rule, 2026-06-20). The boot
disk has
242     > `autoDelete=true`, so delete reclaims it too ? verified 2026-06-20.
243
244     ```bash
245     bash deploy/gcp/teardown.sh          # DELETE the VM (default; $0 ? removes VM + boot
disk)
246     bash deploy/gcp/teardown.sh delete   # same (explicit)
247     bash deploy/gcp/teardown.sh stop     # STOP only ? keeps the disk ? STILL BILLS
(~$/mo!); fast resume
248     bash deploy/gcp/teardown.sh all      # delete VM + SA + budget (keeps the bucket/data)
249     # cross-region VM: set the zone (teardown defaults to asia-south1-a)
250     RALLY_GCP_ZONE=asia-southeast1-a bash deploy/gcp/teardown.sh delete
251     ```
252
253     **After deleting, audit for orphans** (a stray disk bills even with no VM):
254     `gcloud compute instances list` (empty) and `gcloud compute disks list` (no unattached
disk).
255
256     The **bucket is never auto-deleted** (it holds the corpus). Delete it explicitly only
when done:
257     `gcloud storage rm -r gs://kheIsutra-rally-corpus`.
258

```

24. assistant (2026-06-21T14:37:17.826Z)

Now let me look at the compute base to understand the full architecture:

25. user (2026-06-21T14:37:18.746Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
DETECT?WINDOW?STITCH
5     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
6     (cloud-serving) ? without the core ever branching on the profile (D1).
7
8     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
9     non-local backend. Every other target (``"local_gpu" / "cpu_mock"``) keeps the

```

```

worker's
10     existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
inline").
11     """
12
13     from __future__ import annotations
14
15     from abc import ABC, abstractmethod
16     from dataclasses import dataclass, field
17     from typing import Optional
18
19
20     @dataclass(frozen=True)
21     class ComputeJobSpec:
22         """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
(re)compute
23         from the SAME input bytes, so the output always lands at
``<out_uri>/outputs/<video_id>/`` ?
24         deterministic regardless of where the job ran."""
25
26         video_uri: str # input (gs://? for cloud; a path/local://
for local)
27         out_uri: str # output ROOT (gs://? bucket for cloud)
28         segmenter: Optional[str] = None
29         # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
frozen/hashable.
30         config_overrides: tuple[str, ...] = ()
31
32
33     @dataclass(frozen=True)
34     class ComputeResult:
35         """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
code
36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

26. user (2026-06-21T14:37:19.201Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``

```

```

5         otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6         identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7         changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8         ""
9
10        from __future__ import annotations
11
12        import os
13        from typing import Any, Optional
14
15        from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16        from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18        __all__ = [
19            "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
20        ]
21
22
23        def _storage_dict(config: Any) -> dict:
24            storage = getattr(config, "storage", None)
25            if storage is None:
26                return {}
27            if hasattr(storage, "model_dump"):
28                return storage.model_dump() # type: ignore[no-any-return]
29            return dict(storage) if isinstance(storage, dict) else {}
30
31
32        def get_compute_backend(
33            config: Any,
34            *,
35            run_client: Optional[Any] = None,
36            policy: ExecutionPolicy = DEFAULT_POLICY,
37            token: Optional[str] = None,
38        ) -> Optional[ComputeBackend]:
39            """Return the compute backend for ``config.deployment.compute_target``, or ``None``
to run
40            in-process (the default-OFF path).
41
42            Only ``compute_target == "cloud_run"`` returns a non-None backend (a
``CloudRunCompute``). Its
43            Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
``CLOUD_RUN_REGION``
44            / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
injectable (tests
45            pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
verified on M7).
46            ""
47            target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48            if target != "cloud_run":
49                return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51            # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
selected.
52            from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
53
54            job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
55            region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
56            project = os.environ.get("GOOGLE_CLOUD_PROJECT")
57            client = run_client if run_client is not None else GoogleCloudRunJobClient()
58            return CloudRunCompute(
59                run_client=client, job_name=job_name, region=region, project=project,
60                storage_config=_storage_dict(config), policy=policy, token=token,

```

61)
62

27. assistant (2026-06-21T14:37:21.261Z)

Now let me check the job worker implementation to understand in-memory vs persistent state:

28. user (2026-06-21T14:37:22.151Z)

```
1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6      re-claimed when due ? with no central scheduler (the DB is the clock).
7
8      The monkeypatch contract these functions rely on (and that tests depend on):
9      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11     `backend.main` module object at call-time (`import backend.main as _main`), never as a
12     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13     ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15     inside the functions also avoids the circular import at module load (main.py imports
this
16     module; this module must not import main at import time).
17     """
18     import logging
19     import threading
20     import traceback
21     from concurrent.futures import ThreadPoolExecutor
22     from typing import Any, Dict, Optional
23
24     logger = logging.getLogger(__name__)
25
26     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29     # already parallelize their AI handoff internally via their own pools.
30     _job_executor: Optional[ThreadPoolExecutor] = None
31
32
33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38         return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
```

```

50         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
51         """
52
53     def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54         super().__init__(f"job parked for {delay_sec:.0f}s")
55         self.delay_sec = max(0.0, float(delay_sec))
56         self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
74         its
75         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
76
77         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
78         means
79         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
80         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
81         the
82         job self-scheduled and will be re-claimed when `next_poll_at` is due.
83         """
84         import backend.main as _main
85         db_instance = _main.db_instance
86         job_id = job["id"]
87         req = _main._job_to_request(job)
88         try:
89             result = _main._execute_processing(
90                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
91             if result.get("video_id"):
92                 db_instance.set_job_video_id(job_id, result["video_id"])
93             db_instance.mark_job_done(job_id, result)
94             _maybe_record_job_cost(job, result) # M8 cost ledger (default-OFF)
95         except _main.JobWaiting as w:
96             # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
97             logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
98             db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
99         except Exception as e:
100             logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
101             db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")

```

29. assistant (2026-06-21T14:37:24.443Z)

Perfect! Now let me check main.py for the --server flag handling and PORT env binding:

30. user (2026-06-21T14:37:25.472Z)

```

7:import uvicorn
583:     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
584:     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI

```

```

server on")
744:         print(f"[REPORT] Technical report : {paths.get('technical_md')}")
745:         print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
746:         print(f"[REPORT] Machine JSON      : {paths.get('json')}")
756:         uvicorn.run(app, host="127.0.0.1", port=args.port)

```

31. assistant (2026-06-21T14:37:28.025Z)

Excellent. Now let me look at what the CloudRun README says about serving (not just jobs):

32. user (2026-06-21T14:37:28.876Z)

(Bash completed with no output)

33. assistant (2026-06-21T14:37:30.937Z)

Let me look for any existing service dockerfile or related files:

34. user (2026-06-21T14:37:32.052Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/deploy/cloudrun/Dockerfile

35. assistant (2026-06-21T14:37:34.629Z)

Now let me check the storage module to understand if jobs are persisted properly:

36. user (2026-06-21T14:37:35.412Z)

```

244     def _migrate_to_v5(self, cursor):
245         # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
246         # version): the resolved per-user `fusion_config` overlay + an INLINE
247         # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
248         # and provenance that earned it. `is_active` pins the one row the serving bridge
249         # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
250         # only persists/loads it ? NOTHING reads it on the served path yet.
251         #
252         #   user_id           ? owner of the model (NULL allowed = the local install)
253         #   version          ? monotonic per-user model version
254         #   baseline_version ? the shared baseline this was fit against (upgrade
switch)
255         #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
256         #   fusion_config    ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
257         #   classifier_json  ? JSON LogisticRegression.to_dict() (None = config-only
model)
258         #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it was
promoted)
259         #   n_videos_at_fit   ? held-out-user corpus size when fit (min_n trust floor
input)
260         #   is_active        ? the one resolved row for this user (partial-unique
below)
261         cursor.execute("""
262             CREATE TABLE IF NOT EXISTS user_models (
263                 id INTEGER PRIMARY KEY AUTOINCREMENT,
264                 user_id TEXT,
265                 version INTEGER NOT NULL DEFAULT 1,
266                 baseline_version TEXT,
267                 feature_schema_hash TEXT,
268                 fusion_config TEXT,
269                 classifier_json TEXT,
270                 promotion_evidence TEXT,

```

```

271         n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
272         is_active INTEGER NOT NULL DEFAULT 0,
273         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
274     )
275     """
276     # One model version per user (NULLs compare distinct in SQLite, so the local
277     # install can still carry multiple versioned rows; the active-row guard below
278     # is what enforces a single served model).
279     cursor.execute("""
280         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
281         ON user_models(user_id, version)
282     """)
283     # At most ONE active model per user: a partial-unique index over the active
rows.
284     # (SQLite treats NULL user_id rows as distinct, so the local install's single
285     # active row is still guarded ? there is one local user.)
286     cursor.execute("""
287         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
288         ON user_models(user_id) WHERE is_active = 1
289     """)
290     cursor.execute("PRAGMA user_version = 5")
291
292     def _migrate_to_v6(self, cursor):
293         # v6: per-job COST LEDGER (COMPUTE_DECOUPLED_SERVING M8, D8). NULLABLE cost
columns on
294         # `jobs` (NULL = no cost recorded = byte-identical to today; nothing records
until
295         # billing.enabled + a real pricebook) + a VERSIONED `pricebook` table. Cost is
stored in
296         # USD (matches GCP billing = one source of truth); INR is a display conversion
at a
297         # configurable FX rate. ALTER (not recreate) so v1-v5 job rows survive; ADD
COLUMNS are
298         # idempotent so an interrupted upgrade can re-run them. Additive ONLY ? no
served change.
299         for col, typ in (
300             ("owner_id", "TEXT"),           # who pays (the non-owner user / tenant)
301             ("compute_backend", "TEXT"),     # local_gpu | cloud_run | ...
302             ("gpu_type", "TEXT"),            # L4 | T4 | ... (selects the per-second
GPU rate)
303             ("gpu_active_seconds", "REAL"),
304             ("wall_seconds", "REAL"),
305             ("bytes_out", "INTEGER"),        # egress
306             ("gemini_cost_usd", "REAL"),     # provider-reported, already USD
307             ("pricebook_version", "TEXT"),
308             ("cost_usd", "REAL"),            # computed total (USD = source of truth)
309             ("cost_breakdown_json", "TEXT"), # {gpu_seconds: .., egress_gb: ..,
gemini_usd: ..}
310         ):
311             self._add_column_idempotent(cursor, f"ALTER TABLE jobs ADD COLUMN {col}
{typ}")
312         # Partial index keeps the NULL=no-cost fast path churn-free while making per-
owner billing cheap.
313         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_owner ON jobs(owner_id) "
314             "WHERE owner_id IS NOT NULL")
315         # Versioned pricebook: one row per (version, resource, gpu_type). Rates change +
differ by
316         # gpu_type ? re-version (never mutate a shipped version, so a recorded cost
stays auditable).
317         cursor.execute("""
318             CREATE TABLE IF NOT EXISTS pricebook (
319                 version TEXT NOT NULL,
320                 resource TEXT NOT NULL,
321                 gpu_type TEXT NOT NULL DEFAULT '',
322                 unit_price_usd REAL NOT NULL DEFAULT 0.0,

```

```

323             currency TEXT NOT NULL DEFAULT 'USD',
324             effective_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
325         )
326     """)
327     cursor.execute("CREATE UNIQUE INDEX IF NOT EXISTS idx_pricebook_key "
328                   "ON pricebook(version, resource, gpu_type)")
329     # Seed a PLACEHOLDER version with every rate 0.0 ? cost_usd computes to 0
330     (nothing
331     # user-facing changes). The owner INSERTs a real, calibrated version (real GCP
332     $/GPU-s,
333     # egress $/GB) and flips billing.pricebook_version to it ? never overwrite this
334     seed.
335     for resource, gpu_type in (("gpu_seconds", "L4"), ("gpu_seconds", "T4"),
336                               ("wall_seconds", ""), ("egress_gb", "")):
337         cursor.execute(
338             "INSERT OR IGNORE INTO pricebook (version, resource, gpu_type,
339             unit_price_usd, "
340             "currency) VALUES ('placeholder-v0', ?, ?, 0.0, 'USD')", (resource,
341             gpu_type))
342         cursor.execute("PRAGMA user_version = 6")
343
344     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
345     List[Dict[str, Any]]) -> bool:
346         """Register or update a video row WITHOUT touching its child segments.
347
348         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
349         with foreign_keys ON ? cascade-deletes every
350         play_segment/event/candidate_segment.
351         That made a status update (e.g. marking 'failed' on re-validation, or
352         'processed'
353         before the segmenter runs) silently destroy prior good results. UPSERT mutates
354         the
355         row in place; clearing segments is now an explicit, deliberate operation
356         (clear_video_data / prune_segments).
357         """
358         return self._execute_write(
359             "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
360             ?, ?) "
361             "ON CONFLICT(id) DO UPDATE SET "
362             "filepath=excluded.filepath, status=excluded.status, "
363             "validation_results=excluded.validation_results",
364             (video_id, filepath, status, json.dumps(validation_results)),
365             "Failed to add video",
366         )
367
368     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
369         """Return (max play_segment id, max candidate_segment id) for a video (0 if
370         none).
371
372         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
373         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
374         """
375         with self._connect() as conn:
376             cur = conn.cursor()
377             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
378             video_id = ?",
379                             (video_id,)).fetchone()[0]
380             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
381             video_id = ?",
382                             (video_id,)).fetchone()[0]
383             return int(p), int(c)
384
385     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
386                       play_after: Optional[int] = None, cand_upto: Optional[int] =
387                       None,

```

```

374             cand_after: Optional[int] = None) -> None:
375         """Delete play/candidate segments by id range (events cascade from
play_segments).
376
377         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
378         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
379         and keep the OLD ones when the run failed/produced nothing (`*_after`).
380         """
381         with self._connect() as conn:
382             cur = conn.cursor()
383             if play_upto is not None:
384                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
385             if play_after is not None:
386                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
387             if cand_upto is not None:
388                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
389             if cand_after is not None:
390                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
391             conn.commit()
392
393         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:

```

37. assistant (2026-06-21T14:37:38.258Z)

Let me look at the job creation and claim methods in the database:

38. user (2026-06-21T14:37:39.545Z)

```

538:     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
574:     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
592:     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
662:     def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
734:     def claim_due_job(self) -> Optional[Dict[str, Any]]:

```

39. user (2026-06-21T14:37:42.124Z)

```

538         def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
539                         player_pool: Optional[List[str]] = None,
540                         max_frames: Optional[int] = None,
541                         policy: Optional[Dict[str, Any]] = None,
542                         owner_id: Optional[str] = None) -> bool:
543             """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
544             (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
545             (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default)."""
546             return self._execute_write(
547                 "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy, "
548                 "owner_id) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?)",
549                 (job_id, video_path, segmenter,
550                  json.dumps(player_pool) if player_pool is not None else None,
551                  max_frames, json.dumps(policy) if policy is not None else None, owner_id),
552                 f"Failed to create job {job_id}",
553             )
554

```

```

555     def mark_job_running(self, job_id: str) -> bool:
556         return self._execute_write(
557             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
558             "progress='starting' WHERE id = ?", (job_id,),
559             f"Failed to mark job {job_id} running",
560         )
561
562     def set_job_progress(self, job_id: str, progress: str) -> bool:
563         return self._execute_write(
564             "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
565             f"Failed to set progress for job {job_id}",
566         )
567
568     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
569         return self._execute_write(
570             "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
571             f"Failed to set video_id for job {job_id}",
572         )
573
574     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
575         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
576         validation) is inside `result` ? job status 'done' means 'the worker
577         finished'."""
578         return self._execute_write(
579             "UPDATE jobs SET status='done', progress='finished', result=?, "
580             "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
581             (json.dumps(result), job_id),
582             f"Failed to mark job {job_id} done",
583         )
584
585     def mark_job_failed(self, job_id: str, error: str) -> bool:
586         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
587         return self._execute_write(
588             "UPDATE jobs SET status='failed', error=?, "
589             "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
590             f"Failed to mark job {job_id} failed",
591         )
592
593     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
594         """Fetch one job; `result`/`player_pool` JSON deserialized."""
595         with self._connect() as conn:
596             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
597 (job_id,)).fetchone()
598             if not row:
599                 return None
600             d = dict(row)
601             d["result"] = json.loads(d["result"]) if d["result"] else None
602             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
603             None
604             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
605             return d
606
607     # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)
608     -----
609     def get_pricebook_rates(self, version: str, gpu_type: Optional[str] = None) ->
610     Dict[str, float]:
611         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
612         The per-second
613         GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
614         ``gpu_type=''``.
615         An unknown version ? ``{}`` (so every cost computes to 0.0)."""
616         gt = gpu_type or ""
617         rates: Dict[str, float] = {}
618         with self._connect() as conn:
619             rows = conn.cursor().execute(

```

```

613         "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE version
= ?",
614         (version,)).fetchall()
615     for r in rows:
616         res, row_gt, price = r["resource"], (r["gpu_type"] or ""),
float(r["unit_price_usd"])
617         if res == billing.GPU_SECONDS:
618             if row_gt == gt:         # the GPU rate for THIS gpu_type
619                 rates[res] = price
620             else:
621                 rates[res] = price     # wall/egress: gpu_type-independent
622     return rates
623
624     def record_job_cost(self, job_id: str, *, usage: Dict[str, float],
pricebook_version: str,
625                         gpu_type: Optional[str] = None, compute_backend: Optional[str] =
None,
626                         owner_id: Optional[str] = None, margin: float = 0.0) ->
Optional[Dict[str, Any]]:
627         """Compute the job's cost from ``usage`` x the pricebook + persist every cost
column.
628         Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
629         ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
630         rates = self.get_pricebook_rates(pricebook_version, gpu_type)
631         # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
632         # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
633         # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
634         gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
635         unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
636         if unpriced_gpu:
637             logger.warning(
638                 "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
639                 "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix
gpu_type.",
640                 gpu_type, pricebook_version, gpu_amt)
641         cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
642         if unpriced_gpu:
643             breakdown["unpriced_gpu_seconds"] = gpu_amt
644         if margin:
645             marked = billing.apply_margin(cost_usd, margin)
646             breakdown["margin"] = round(marked - cost_usd, 6)    # keep ?(breakdown) ==
cost_usd
647             cost_usd = marked
648         bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
649         ok = self._execute_write(
650             "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
651             "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
652             "cost_breakdown_json=? WHERE id = ?",
653             (owner_id, compute_backend, gpu_type, usage.get(billing.GPU_SECONDS),
usage.get(billing.WALL_SECONDS), bytes_out, usage.get(billing.GEMINI_USD),
654             pricebook_version, cost_usd, json.dumps(breakdown), job_id),
655             f"Failed to record cost for job {job_id}")
656         if not ok:

```

40. assistant (2026-06-21T14:37:44.805Z)

Perfect! Now let me check the claim_due_job to see how jobs are claimed:

41. user (2026-06-21T14:37:46.063Z)

```
734     def claim_due_job(self) -> Optional[Dict[str, Any]]:
735         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
736         ``next_poll_at`` is due ?
737         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
738         never
739         double-claim (the DB row-claim is the only coordination needed; no master).
740         Earliest-due
741         first. Returns the claimed job dict, or None if nothing is runnable."""
742         try:
743             with self._connect() as conn:
744                 cur = conn.cursor()
745                 row = cur.execute(
746                     "SELECT id, status FROM jobs WHERE status='queued' "
747                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
748                     CURRENT_TIMESTAMP)) "
749                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
750                     1").fetchone()
751                 if not row:
752                     return None
753                 cur.execute(
754                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
755                     "WHERE id=? AND status=?", (row["id"], row["status"]))
756                 conn.commit()
757                 if cur.rowcount != 1:
758                     return None # lost the race to a concurrent
759                 worker
760                 return self.get_job(row["id"])
761         except Exception as e:
762             logger.error(f"Failed to claim a due job: {e}")
763             return None
764
765     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
766         """Crash recovery: any job still queued/running from a previous process is dead
767         (the executor is in-process). Mark failed so clients see a terminal state, not a
768         hang. Called once at startup. Returns the number of jobs swept."""
769         try:
770             with self._connect() as conn:
771                 cur = conn.cursor()
772                 cur.execute(
773                     "UPDATE jobs SET status='failed', error=?,
774                     finished_at=CURRENT_TIMESTAMP "
775                     "WHERE status IN ('queued', 'running')", (reason,))
776                 conn.commit()
777                 return cur.rowcount
778         except Exception as e:
779             logger.error(f"Failed to sweep stale jobs: {e}")
780             return 0
781
782     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
783     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
784     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
785     # serving.py) resolves them, and wiring that bridge into the production decision
786     seam
787     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
788     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
789     # JSON-serialised dicts; the rest are scalar columns.
790
791     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
792                           baseline_version: Optional[str] = None,
793                           feature_schema_hash: Optional[str] = None,
794                           fusion_config: Optional[Dict[str, Any]] = None,
795                           classifier_json: Optional[Dict[str, Any]] = None,
796                           promotion_evidence: Optional[Dict[str, Any]] = None,
```

```

789             n_videos_at_fit: int = 0,
790             is_active: bool = False) -> Optional[int]:
791         """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.
792
793         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
794         the
795         rowid). When ``is_active`` is True this row becomes the user's single active
796         model ?
797         any previously-active row for the SAME user is first cleared, so the
798         ``idx_user_models_one_active`` partial-unique invariant always holds (one active
799         model
800         per user). Returns the row id, or None on failure.
801         """
802         try:
803             with self._connect() as conn:
804                 cur = conn.cursor()
805                 if is_active:
806                     # Demote the current active row first (NULL user_id = local: IS NULL
807                     match).
808                     if user_id is None:
809                         cur.execute(
810                             "UPDATE user_models SET is_active = 0 "
811                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
812                             (version,))
813                     else:
814                         cur.execute(
815                             "UPDATE user_models SET is_active = 0 "
816                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
817                             (user_id, version))

```

42. assistant (2026-06-21T14:37:48.192Z)

Perfect! Now let me verify the db_path in main.py to confirm SQLite persistence:

43. user (2026-06-21T14:37:49.326Z)

```

659: db_path = os.path.join(global_config.output.output_dir, global_config.output.db_name)
660: db_instance = Database(db_path)
732: print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")

```

44. assistant (2026-06-21T14:37:51.340Z)

Now let me check if there's any documentation about the deploy story:

45. user (2026-06-21T14:37:52.051Z)

```

C:\Users\avidu\Projects\badminton-highlight-indexer\docs\ABLATION_LAYER.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\ANALYZERS_AND_RECONCILERS.md
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\checkpoints\2026-06-14-hardening-loop-complete.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-eval-
baseline\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-logging-
reporting\README.md
C:\Users\avidu\Projects\badminton-highlight-
indexer\docs\archives\checkpoints\2026-06-scaffolding-complete\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\checkpoints\2026-06-strategy-
foundation\README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\CODE_AUDIT_AND_TEST_HARDENING-
COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\COMMERCIAL_READINESS_REVIEW.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\decisions\DECISIONS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs\archives\DEFERRED_HARDENING_PLAN-

```

COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/FIELD_VISIT_ANSWER_SHEET.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/FIELD_VISIT_PLAYBOOK.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/PMF_FIELD_SIGNALS.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/PMF_MARKET_RESEARCH.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/PRODUCT_FIELD_ASSOCIATE_FLYER.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/field-pmf/README.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/HARDENING_LOOP_HANDOFF-COMPLETED-2026-06-14.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/MONETIZATION_AUDIT.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md

46. assistant (2026-06-21T14:37:54.460Z)

Let me search for any deploy or hosting documentation:

47. user (2026-06-21T14:37:55.165Z)

C:\Users\avidu\Projects\badminton-highlight-indexer\docs/archives/past_projects/DECOUPLED_COMPUTE/DEPLOY_REPORT_GCP_2026-06-20.md
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/COMPUTE_DECOUPLED_SERVING
C:\Users\avidu\Projects\badminton-highlight-indexer\docs/HOSTING_PLAN.md

48. user (2026-06-21T14:37:57.727Z)

```
1      # Hosting Plan ? khelsutra.guru
2
3      **Status:** PROPOSAL ? awaiting owner ratification.
4      **Date:** 2026-06-10 . **Constraints:** budget $10?25/mo . GCP credits available .
DigitalOcean
5      acceptable . alpha backend = owner's GPU box behind a tunnel . 10?25 alpha testers .
registrar = GoDaddy.
6
7      > **? Update (2026-06-20, ADR-013): DigitalOcean is DROPPED for COMPUTE** ? GCP is the
sole compute
8      > stack (training + serving). The ***"All-DigitalOcean"*** compute option below is
retired; DO Spaces
9      > remains only a *possible* S3-compatible storage target. Edge/site hosting choices here
are unaffected.
10     **Companion:** `docs/UI_PROPOSAL.md`.
11
12     ## 1. Recommendation in one line
13
14     **Alpha: free edge + your box** (Cloudflare DNS/Pages/Access/Tunnel ? FastAPI on the GPU
box) at
15     **$0/mo**, saving GCP credits; **Beta: GCP control plane + your box as pull-worker**
(Cloud Run +
16     GCS + Firebase Auth, credits-covered), keeping the edge where it's free.
17
18     Rationale for the Cloudflare pieces despite the GCP preference: GCP has no free
equivalent of the
19     Tunnel + Access combo (IAP requires a load balancer, ~$18/mo standing cost), and the
alpha needs
20     exactly three things GCP charges for but Cloudflare gives free: a secure tunnel to a
home box with
21     a custom domain, email-allowlist auth (free ?50 users ? fits 10?25 testers), and static
hosting.
22     Credits are better spent at beta on compute/storage, which is where GCP is excellent.
23
```

```

24    ## 2. Alpha topology ($0/mo)
25
26    ```
27    tester browser
28        ? https
29        ?
30    Cloudflare edge (free): DNS + TLS + Access (email-OTP allowlist)
31        ??? app.khelsutra.guru ? Cloudflare Pages (React SPA; auto-deploy from GitHub)
32        ??? api.khelsutra.guru ? Cloudflare Tunnel (outbound-only `cloudflared` service on
the box)
33
34        ?
35        ?
36        FastAPI :8000 on the GPU box (Windows)
37        ? Gemini segmenter path (no GPU in request path)
38        ? async jobs (#16), SQLite WAL, output/ on disk
39    ```
40    - **No inbound ports opened** on the home network; `cloudflared` dials out. Home IP
never exposed.
41    - **Auth:** Cloudflare Access in front of BOTH hostnames; backend trusts the
42        `Cf-Access-Authenticated-User-Email` header (verify the Access JWT for rigor) for per-
user scoping.
43    - **Upload limit:** free-tier proxy caps request bodies at **100 MB** ? SPA uploads in
?90 MB
44        chunks (UI_PROPOSAL B2). Match videos (1?4 GB) flow fine as chunks.
45    - **Box offline = service offline.** Acceptable for alpha; SPA shows a friendly
"processing engine
46        offline" banner (a `/api/health` ping). Beta removes this with the queue.
47    - **Costs:** Pages, Tunnel, Access, DNS = $0. Only the Gemini per-run cost
(~$0.05/match) and
48        domain renewal.
49
50    ## 3. Beta topology (GCP credits; ~$15?20/mo post-credits)
51
52    ```
53    browser ??? Cloudflare edge (free, unchanged) ??? app.khelsutra.guru (Pages)
54        ??? api.khelsutra.guru ??? Cloud Run
(FastAPI control plane, scale-to-zero)
55
56        ? jobs
table (Cloud SQL Postgres)
57
58        ? signed
resumable URLs
59
60        ?
61        GCS buckets
(uploads / reels, lifecycle ? cold)
62
63        ? pull
jobs, outbound-only
64
65        GPU box = first
`byo_worker` (PLATFORM P3)
66    ```
67
68    - Uploads go **browser ? GCS directly** (resumable signed URLs) ? no proxy body limit,
no home
69    bandwidth for ingest; the box pulls only what it processes (WASB/owned-model jobs),
Cloud Run
70    handles Gemini-path jobs entirely in-cloud.
71    - Auth graduates to **Firebase Auth** (free; Google + email sign-in; India-friendly
phone OTP
72    later); Access allowlist retires.
73    - Post-credits estimate: Cloud Run ~$0?5 (free tier: 2 M req + 180 K vCPU-s/mo, scale-
to-zero) .
74    GCS 100 GB ? $2.60 + egress $0.12/GB (reel downloads ? watch this line) . Cloud SQL
smallest
75    ? $10 ? **~$15?20/mo**, inside budget. Egress lever if it grows: serve reels from
Cloudflare R2

```

```

71         (zero egress) while keeping everything else GCP.
72
73     ## 4. Alternatives considered
74
75     | Option | Monthly (alpha ? beta) | Pros | Cons |
76     |---|---|---|---|
77     | **Recommended hybrid** (CF edge + box ? +GCP) | **$0 ? ~$0 w/ credits** (~$15?20
after) | Cheapest at every phase; credits used where they matter; maps to PLATFORM P2/P3 | Two
vendors (CF + GCP) |
78     | **All-DigitalOcean** | ~$11 ? ~$17 ($6 droplet + Spaces $5: 250 GiB + **1 TiB egress
included**) | One simple vendor; Spaces' included egress is the best video-download deal; per-
second billing | No credits; no scale-to-zero; still needs a tunnel-equivalent to reach the box
(run `frp`/WireGuard on the droplet) |
79     | **All-GCP from day 1** | ~$18+ ? credits | Single vendor, credits immediately | LB+IAP
standing cost just to gate an alpha; no free tunnel to the home box; credits burn on plumbing |
80     | **All-Cloudflare** (Pages+Workers+R2) | ~$0 ? ~$5 | Zero egress on R2; cheapest
forever | FastAPI doesn't run on Workers (API stays on box/Cloud Run anyway); ignores GCP
credits |
81
82     ## 5. Domain setup ? GoDaddy ? Cloudflare DNS (one-time, ~15 min, you)
83
84     1. Create a free Cloudflare account ? **Add a domain** ? `khelsutra.guru` ? Free plan.
Cloudflare
85         imports existing records and shows **two nameservers** (e.g.
`xxx.ns.cloudflare.com`).
86     2. GoDaddy ? **My Products ? khelsutra.guru ? Manage DNS ? Nameservers ? Change ?
87         "I'll use my own nameservers"** ? paste the two Cloudflare nameservers ? Save.
Propagation:
88         minutes to 24 h. (Registration/renewal stays at GoDaddy; only DNS moves. Optional
later:
89         transfer the registration to Cloudflare Registrar for at-cost .guru renewals.)
90     3. Records (I manage after takeover): `app` ? Pages project; `api` ? `**Cloudflare API token** (you create: dash ? My Profile ? API Tokens) | Zone:DNS>Edit,
Account:Pages>Edit, Account:Access:Apps>Edit, Account:Tunnel>Edit ? scoped to this zone | DNS
records, Pages deploys (wrangler), Access policies, tunnel config |
98     | **GitHub repo** `khelsutra` (private) | Pages connects once (dashboard, you; build
root = `web/`) ? every push auto-deploys | I write the SPA in your Projects folder; commits
pushed = deployed |
99     | **On the box** (one-time, I hand you exact commands) | `cloudflared` installed as a
Windows service + `config.yml` | The tunnel itself |
100    | **GCP (beta only)** | A `khelsutra` project + service account (Cloud Run admin,
Storage admin, Cloud SQL admin) | Control-plane deploys |
101
102    Day-to-day flow after setup: I build ? commit ? push ? Pages deploys the SPA
automatically;
103    API/tunnel/DNS changes via the scoped token. Secrets (Gemini key, Access policy) never
live in the
104    repo; `.env` on the box + Cloudflare/GCP dashboards.
105
106    ## 7. Go-live security checklist (alpha)
107
108    Access allowlist ON for both hostnames ? CORS narrowed to `https://app.khelsutra.guru` ?
rate
109    limit + per-user upload quota ? `security.allowed_video_root` set ? upload size cap +
extension
110    allowlist ? **rotate the Gemini API key** (NEXT_STEPS housekeeping ? it was shared in
chat) ?
111    confirm no router port-forwards ? weekly backup: SQLite DB + `output/` ? GCS bucket

```

```
(rclone
112     scheduled task; credits).
113
114     ## 8. Decisions needed from owner
115
116     1. Ratify the hybrid recommendation (CF free edge now, GCP credits at beta)? Or
force
117         single-vendor (DO / GCP) despite the cost notes above?
118     2. OK to move DNS to Cloudflare (registration stays at GoDaddy)?
119     3. Beta reel storage: GCS (credits) vs R2 (zero egress) vs DO Spaces (1 TiB
included) ?
120         can defer until beta Drop 2.
121     4. Approve the access grants in S6 so I can run the setup end-to-end.
122
123     *Ratified choices get recorded in `docs/archives/decisions/DECISIONS.md`.*
124
```

49. assistant (2026-06-21T14:38:01.030Z)

Excellent! Now I have all the context I need. Let me create a comprehensive summary by compiling all my findings:

50. user (2026-06-21T14:39:02.496Z)

Structured output provided successfully

51. assistant (2026-06-21T14:39:19.755Z)

Perfect! I've completed a thorough analysis of the Cloud Run deploy story. Let me provide a final summary for context:

Cloud Run Deploy Story ? Complete Mapping

Current State (The Per-Request Job Worker Path)

The codebase **successfully deploys the GPU Job worker** to Cloud Run:

- **deploy/cloudrun/Dockerfile** builds a CUDA 12.4 image with the WASB conda environment
- **CloudRunCompute** (backend/compute/cloud_run.py) dispatches inference jobs to Cloud Run GPU Jobs
- Completion signals via bucket-pollled done-marker (storage-backed, not Cloud Run API)
- Verified on real L4 run (M7, 2026-06-20, \$100 budget)

Missing: FastAPI Service Deployment

To reach the goal of a **persistent, internet-reachable `/api/process` endpoint**, five critical gaps exist:

1. **No Service Dockerfile** ? Only the worker image exists. Need a lightweight CPU image that runs `python -m backend.main --server` with uvicorn.`
2. **main.py binds to 127.0.0.1 only** ? Cloud Run requires `HOST=0.0.0.0` + `PORT` env var.` Line 756 is hardcoded; no env fallback.
3. **No `gcloud run deploy` command for the Service** ? Only Job creation documented. Service needs memory, CPU, timeout, and env var wiring.
4. **SQLite database is ephemeral** ? Output file lives in local `/workspace``, which resets on each instance spawn. Multi-instance scale-to-zero loses job state unless migrated to Cloud SQL or a persistent volume.
5. **No secrets/auth integration** ? WASB weights, Gemini key, GCP project must be wired via Secret Manager or env; currently manual.

What Works Today

? ****Durable job state****: SQLite persists jobs with atomic ``claim_due_job()`` for concurrent workers
? ****API endpoints****: All required routes exist (POST `/api/process`, GET `/api/jobs`, `/api/compile`, `/api/highlights`)
? ****Health check****: GET `/api/health` returns service status
? ****Async execution****: `ThreadPoolExecutor(max_workers=1)` drains jobs; `JobWaiting` self-schedules parked jobs
? ****Cloud-serving profile****: ``deployment.compute_target=cloud_run`` gates the Job dispatch shim

Classification

- ****Dockerfile + PORT fix****: claude-doable (code changes only)
- ****gcloud run deploy script****: claude-doable (documented pattern)
- ****SQLite ? Cloud SQL migration****: owner-gated (GCP spend + data migration)
- ****Load balancer + HTTPS****: owner-gated (standing cost, DNS/cert mgmt)
- ****Actual `gcloud run deploy` execution****: owner-gated (requires active GCP project + spend approval)